



VRIJE
UNIVERSITEIT
BRUSSEL



Master thesis submitted in partial fulfilment of the requirements for the degree of
Master of Science in de Ingenieurswetenschappen: Computerwetenschappen

GROUNDING FIRST-ORDER LOGIC OVER FINITE DOMAINS USING A RELATIONAL DATABASE

Nigel Vinckier

2025 – 2026

Supervisor: Prof. Dr. Joost Vennekens
Science and Bio-Engineering Sciences



VRIJE
UNIVERSITEIT
BRUSSEL



Proefschrift ingediend met het oog op het behalen van de graad van Master of Science in de Ingenieurswetenschappen: Computerwetenschappen

GROUNDING VAN EERSTE-ORDE-LOGICA OVER EINDIGE DOMEINEN GEBRUIKMAKEND VAN EEN RELATIONELE DATABANK

Nigel Vinckier

2025 – 2026

Promotor: Prof. Dr. Joost Vennekens

Wetenschappen en Bio-ingenieurswetenschappen

Abstract

This master's thesis is about grounding a theory in the context of satisfiability problems. These are problems formulated in the language of propositional logic, which can be handed to a state-of-the-art solver. If, however the theory is formulated in first-order logic, it needs to be translated to propositional logic by eliminating quantifiers, a process called *grounding*.

Over finite domains, the naive grounding method is to convert a universally quantified formula into a conjunction, and an existentially quantified one into a disjunction. The naive approach leads to formulas with a lot of redundancies, which the solver needs to weed out, which can make the solving process slow. The aim of this project is to avoid this redundancy.

In order to do so, a relational database will be used to store (parts of) a structure, and this database will be queried to ground different kinds of formulas. Both the theoretical framework for this and the Python implementation which has been developed, will be explained. After this, an in-depth analysis will be performed of different test cases – both existing benchmarks and newly developed ones – to which the system has been applied.

Contents

Abstract	3
Acknowledgments and Personal Statement	9
1 Introduction	11
1.1 First-Order Logic over Finite Domains	11
1.2 Relational Databases	12
2 Preliminaries	15
2.1 Conventions	15
2.2 SAT Problems	16
2.3 IDP-Z3 and the FO[.] Language	16
2.4 Grounding	17
2.5 SMTLIB	18
2.6 RDBMSs	18
2.7 The Goal of the Thesis	18
3 The IDP-Z3-DB System	21
3.1 Parsing	21
3.2 The Database	22
3.2.1 Building the Database: Vocabulary	22
3.2.2 Building the Database: Structure	23
3.2.3 Consulting the Database: Finding Satisfying Sets	24

3.2.4	Updating the Database: Fact Queries	25
3.3	Grounding of Formulas	27
3.3.1	Formulas without Quantifiers	28
3.3.2	Formulas with Only Universal Quantifiers	28
3.3.3	Formulas with Only Existential Quantifiers	30
3.3.4	Formulas with Mixed Quantifiers: $\forall \exists$	31
3.3.5	Formulas with Mixed Quantifiers: $\exists \forall$	33
3.3.6	Formulas with Mixed Quantifiers: In General	35
3.3.7	Formulas not in Prenex Normal Form	37
3.4	The Solver	38
4	Implementation	39
4.1	Parsing	39
4.2	Building the Database	40
4.3	Grounding the Theory	40
4.4	The Solver	42
4.5	Structure of the Git Repository	43
5	Testing and Comparing	45
5.1	External Comparison	45
5.1.1	DIRT	45
5.1.2	Results and Interpretation	47
5.1.3	Overall Conclusion of the DIRT Benches	51
5.2	Internal Comparison	52
5.2.1	Setting Up the Test Cases	52
5.2.2	Results and Interpretation	53
5.2.3	Overall Conclusion of the Internal Comparison	63
6	Conclusion	67

<i>CONTENTS</i>	7
6.1 Improvements and Extensions	67
6.2 Final Conclusions	68
Bibliography	69

Acknowledgments and Personal Statement

Personal Statement

My path in computer science and the conception of this master's thesis, can be considered somewhat unconventional. After having completed my master's degree in mathematics in 2014 and a postgraduate programme in logic, history and philosophy of science and mathematics in 2015, both at Ghent University, I dabbled with research in philosophy of mathematics at the VUB. At the same time, I started to dig into this unexplored fascination I had with programming. Unhappy with the online resources, I decided to study this more formally by starting the preparatory programme in computer science in 2017, and the master's programme in 2020.

Studying whilst in parallel working full-time hasn't always been the easiest, but I am grateful for the support and flexibility the VUB and its lecturers offered to working students. The most difficult has undoubtedly been the decision to give up the thesis I had been working on during the previous academic year – on a completely different topic: ethics and AI, having quit philosophy research I should have known better when picking the subject... – and restart the work on the thesis currently before the reader. I wish to nonetheless thank my supervisors prof. Loeckx and dr. Arco García and apologise for the error in judgement on my behalf.

The question I have had to answer most over the last 9 years has been: Why? Why continue pursuing another degree? Not for professional reasons; I teach mathematics and programming at university and while my growing computer science knowledge has certainly helped, I did not want to obtain this degree to change jobs or to advance my career. Not for the honours of the degree or to prove something to anyone (including myself); While not unsusceptible to vanity, I am impervious to this one. There is but one sole reason: an appetite for learning. It is a hobby project gotten out of hand. Intellectual masturbation in a time when knowledge for the sake of knowledge is frowned upon. A rejection of the neoliberal premise that academic endeavours should be functional, applicable, career-driven. Why? Because it has been, above all, *fun*.

Acknowledgments

I firstly want to thank my thesis supervisor Joost. The triweekly meetings we had and his support in guiding the research proved vital for the progress of the project. I thank Simon Vandeveldel for the occasional online meeting we had, for providing a *treasurer* starting project and reading parts of the thesis during its conception, just because he was interested in the project. Also Lucas Van Laer helped me a couple of times when stuck. Further, I thank Stijn Vansummeren, with whom I had a meeting in the context of my research internship, during which he suggested

using a different database management library, and gave me more insight in the complexity and execution of queries. Finally, I thank my girlfriend Lison for her support. Without love, this appetite for learning would be but a hollow endeavour.

On the Use of GenAI

If one were to take up art class, one would rely on certain technology, such as using the satnav to drive there. But once there, one would not perform colouring by number, but instead expand one's artistic skills through tuition and struggle. Similarly, in this project, GenAI was only used to explore how to tackle menial Python and SQL tasks, e.g. generating regular expressions or getting to grips with certain modules.¹ No GenAI, however, was used to conceive or implement the concepts presented, nor for the writing of this report. The best guarantee for this is the fact that, as stated above, taking up studying computer science and finishing this master thesis are a “hobby project gotten out of hand”. I used the satnav to get there, I did not colour by number.

The Structure of this Thesis

In Chapter 1, the equivalence between a structure in first-order logic over finite domains and relational databases is explained. Chapter 2 introduces the conventions observed in this thesis, SAT problems and grounding, and states the goal of this project. In Chapter 3, the conceptual side of the system built² is explained in detail, and the logic which proves the approach is correct is demonstrated. Its Section 3.3 can be considered as the heart and main contribution of this thesis. Technical details and design choices of the Python implementation are discussed in Chapter 4. The approach to test the system and the results thereof are described in Chapter 5, after which follows Chapter 6, stating a conclusion.

¹Not different than using a search engine, and in compliance with article 118§2 of the VUB teaching and examination regulations.

²Which can be consulted at <https://github.com/nvinckie/Master-Thesis>. Also refer to Section 4.5, where its structure is detailed.

Am I jumping the gun, Baldrick, or are the words “I have a cunning plan” marching with ill-deserved confidence in the direction of this conversation?

Edmund Blackadder

1

Introduction

1.1 First-Order Logic over Finite Domains

In this thesis, we will deal with typed first-order logic (FOL), where all the domains involved are finite. We will assume the reader is familiar with the basic concepts of FOL¹ and will use the following terminology:

- *Vocabulary*: the names of types², functions and predicates (and for types, their enumeration);
- *Structure*: enumerations for certain predicates;
- *Theory*: formulas which express constraints that need to hold.

As a running example, we will use an instance of the famous map-colouring problem [Vandeveld, 2020], in which the assignment is to colour countries that share a border with a different colour. The *vocabulary* of this problem has two types, namely Colour and Country:

Colour := {Green, Red, Yellow, Orange},

Country := {Belgium, France, Germany, Luxembourg, Netherlands}.

It also has a predicate: *bordering* (expressing which countries border each other) and a function *colour_of_country*: Country → Colour, mapping countries to the colour they have.

The *structure* consists of an enumeration for the *bordering* predicate:

bordering := {(Belgium, France), (Belgium, Luxembourg), ... }.

¹Such as predicates, connectives, quantifiers, functions etc.

²Terms like *set*, *domain* and *type* can be used interchangeably.

There is but one formula in the *theory*, expressing that countries that border, should be coloured differently:

$$\forall x, y \in \text{Country: } \text{bordering}(x, y) \Rightarrow (\text{colour_of_country}(x) \neq \text{colour_of_country}(y)).$$

As will be explained later³, the aim of this thesis will be to convert a set of FOL formulas into a set of equivalent formulas from which all quantifiers have been eliminated, which can be presented to a SAT solver.

In this case, the formula above would be equivalent to

$$\begin{aligned} & \text{bordering}(\text{Belgium}, \text{Belgium}) \Rightarrow \text{colour_of_country}(\text{Belgium}) \neq \text{colour_of_country}(\text{Belgium}) \\ \wedge & \text{bordering}(\text{Belgium}, \text{France}) \Rightarrow \text{colour_of_country}(\text{Belgium}) \neq \text{colour_of_country}(\text{France}) \\ \wedge & \dots \\ \wedge & \text{bordering}(\text{Nether.}, \text{Nether.}) \Rightarrow \text{colour_of_country}(\text{Nether.}) \neq \text{colour_of_country}(\text{Nether.}), \end{aligned}$$

a conjunction over all 25 combinations of the 5 countries. Because we know the bordering predicate entirely⁴, a lot of the implications hold trivially, as their antecedent is false. The conjunction will therefore reduce to

$$\begin{aligned} & \text{colour_of_country}(\text{Belgium}) \neq \text{colour_of_country}(\text{France}) \\ \wedge & \dots \\ \wedge & \text{colour_of_country}(\text{Germany}) \neq \text{colour_of_country}(\text{Luxembourg}), \end{aligned}$$

i.e. a conjunction over those pairs of countries that actually border to each other, as stipulated by the bordering predicate.

1.2 Relational Databases

To perform the task of eliminating quantifiers more efficiently, a key insight and the inspiration for the thesis proposal by Vennekens, is that the structure of problem formulated in FOL over finite domains, can be entirely represented by a relational database.

The types involved translate to a simple table, with their name as primary key (PK).⁵ For the map-colouring example, this yields:

<u>Colour</u>	<u>Country</u>
Green	Belgium
Red	France
Yellow	Germany
Orange	Luxembourg
	Netherlands

³Refer to Section 2.7.

⁴Later we will call this a *fully interpreted predicate*.

⁵For these and tables further, we use the convention of underlining the attribute(s) which form(s) the PK.

Predicates come in two flavours: *fully* versus *partially* interpreted. Fully interpreted predicates describe fixed information about the world, e.g. which countries border each other in this example. These predicates can be represented by an associative table, where the columns are foreign keys (FK) to their respective types:

bordering

<u>Country1</u>	<u>Country2</u>
Belgium	France
Belgium	Luxembourg
...	...

Because the information provided by fully interpreted predicates is total (we know for all pairs of countries whether they border or not), only those pairs of countries bordering need to be listed; if a pair of countries is not mentioned in the table, they do not border. Logical questions can then be answered by means of an SQL query. The following query would yield all countries bordering Belgium:

```
SELECT Country2 FROM bordering WHERE Country1 = "Belgium"
```

Note that also universal (Do all countries border Belgium?) and existential (Are there countries that border Belgium?) questions can be answered this way.

Partially interpreted predicates require a different approach. Their truth value is not (yet) determined, and the goal will often be to find an assignment for their truth values, or check whether such an assignment is possible at all. The same holds for functions, whose function values have not been established (or at least not all). E.g. the aim of the map-colouring problem, is to find a (possible) assignment for the function `colour_of_country`, thus mapping every Country to a Colour.

As will be explained in more detail in Section 3.2.1, we will, for partially interpreted predicates and functions, consider associative tables with an extra column, which represents the truth value of the statement. Note that the interpretation of “not mentioned in the table” is different than for fully interpreted predicates. For the latter, if a tuple of constants doesn’t occur in the table, the predicate doesn’t hold on those. For a partially interpreted predicate or function, this means that for this tuple, the truth value or function value hasn’t been established (yet). The table below illustrates this for the function `colour_of_country`, in case we know Belgium is coloured Green, France is *not* Yellow, and all other assignments are yet undetermined.

colour_of_country

<u>Country</u>	<u>Colour</u>	truth
Belgium	Green	1
France	Yellow	0

Again, questions can be answered by means of a query. The following yields all countries coloured Green:

```
SELECT Country FROM colour_of_country WHERE Colour = "Green" AND truth = 1
```

Now assume that all countries have been assigned a colour, so the table above contains, for every country, a 1 to pair it with a particular colour. Then the constraint of the formula forbidding the same colour for bordering countries, can be translated to the following query:

```
SELECT Country1, Country2 FROM bordering
WHERE (SELECT Colour FROM colour_of_country
       WHERE Country = Country1 AND truth = 1)
      = (SELECT Colour FROM colour_of_country
       WHERE Country = Country2 AND truth = 1)
```

If this query yields any results, two countries that border each other have the same colour and the constraint is violated.

Die Welt ist alles, was der Fall ist.

Ludwig Wittgenstein

2

Preliminaries

2.1 Conventions

Throughout this thesis, we will adopt the following conventions. Capital letters A, B, C, X, Y will represent types (or, equivalently, *sets* or *domains*), lower case letters with an index constants corresponding to the respective type, e.g. $a_1 \in A, x_n \in X$. We will use capital letters P, Q, R etc. for predicates which are fully interpreted in the structure, and therefore represent facts known about the universe. Lower case letters p, q, r etc. are reserved for partially interpreted predicates, f, g, h etc. for functions. Greek lowercase letters φ, ψ, χ etc. are used for logical formulas.

A vector, e.g. $\vec{x}$, is used for a tuple of variables or constants, the ranges of which are left unmentioned if made clear by the context (e.g. $p(\vec{x})$ for a predicate we know to be $p \subseteq A \times B$). We omit the domains if they are clear or irrelevant in the context, and write e.g. $\forall \vec{x} : \varphi(\vec{x})$ instead of $\forall \vec{x} \in A \times B : \varphi(\vec{x})$ or $\forall x \in A : \forall y \in B : \varphi(x, y)$.

For code, input from an IDP file, SMTLIB formulas and SQL queries, the `typewriter` font will be used. In the text, however, types and formulas will be written in the familiar mathematics font, with the usual conventions for FOL notation (e.g. $\wedge$ for a conjunction etc.).

We honour the somewhat bizarre writing style of using the *pluralis auctoris* and opted for British English spelling.¹ Abbreviations are introduced after their first full use and a full list is presented below.

¹The reader will be pleased to learn that this pedantic insistence backfired a couple of times when implementing the map-colo(u)ring problem...

Abbreviation	Explanation
FK	Foreign Key
FOL	First-Order Logic
IDP	Imperative Declarative Programming
PL	Propositional Logic
PK	Primary Key
PNF	Prenex Normal Form
RDBMS	Relational DataBase Management System
SAT	SATisfiability or SATisfiable
UNSAT	UNSATisfiable

2.2 SAT Problems

In the field of satisfiability (SAT) problems, typically, a theory of propositional logic (PL) is described. This often involves a description of a fixed situation, also called a *knowledge base*. In the familiar map-colouring problem, these are the colours, countries and which countries border each other. Additionally, there are logical formulas, which usually state constraints that ought to be satisfied, e.g. bordering countries need to be coloured differently. Note that this formula is described in FOL, not PL, so before this theory can be treated as a SAT problem, the quantified formulas will have to be converted to PL ones. This is the problem of grounding (refer to Section 2.4). The objective of SAT solving is usually to find a model (or all models) for this theory that satisfies these constraints or find out whether such a model exists, and the theory is called *satisfiable* (SAT).² This is so-called *model expansion*³: a *partial* model, consisting of the initial constraints of the fully interpreted predicates (e.g. the bordering predicate in the map-colouring problem) is expanded to a *total* model of the theory, in which all predicates and functions have an interpretation over their entire domains (e.g. finding an interpretation for the function the colour_of_country satisfying the “different colour” constraint). At its (very) worst, it can be shown [Cook, 1971] one has to try all possible variable assignments, and see whether this particular assignment satisfies the constraints, which, for the small version of our map-colouring problem, would boil down to trying all $5^4 = 625$ combinations of country-colour. Since the last decades of the 20th century, there has been a quest for more efficient solvers [Zhang and Malik, 2002], complete with its own [SAT Competitions].

2.3 IDP-Z3 and the FO[.] Language

One of these SAT solvers⁴ is Z3, developed by Microsoft Research. It has its own Python integration and will – as this thesis is not about building a SAT solver but more about the problem of grounding, see further in this chapter – be used “as is”.

²We refer to [Biere et al., 2009] for a more exhaustive introduction.

³Refer to e.g. Carbone [2026]. We state their definition in our terminology.

⁴Actually an SMT solver (Satisfiability Modulo Theories, an extension of SAT problems to include numbers, lists, strings etc.).

The [IDP-Z3] system makes use of this Z3 solver. It supports *Imperative Declarative Programming* (IDP), as introduced by De Cat et al. [2018], which is an extension of classical logic.⁵ Making its grounding algorithm more efficient was the starting point of the thesis proposal. IDP-Z3 makes use of the FO[.]⁶ [Carbonelle and Denecker, b] language to state the logical theory. This language is too rich to cover entirely in a thesis project, however. Some of its subparts support infinite domains⁷ and are inherently incompatible with the finite domains restriction of this project. Some parts of it are not supported given the limited scope of a master's thesis, although there is no a priori reason why the system presented here could not be extended to this. Inequalities and aggregates (min/max/sum) are good examples. We will, therefore, concentrate on FO[Core][Carbonelle and Denecker, a]: multi-sorted logic with finite data structures, functions and equality.

An FO[.] theory is presented to the system in an IDP file⁸. While the (parsing of) different parts of this file will be discussed in more detail in Chapter 3, it is relevant to know its three main parts, which correspond to the terms introduced in Section 1.1:

- A *vocabulary* block, in which types are enumerated and functions and predicates are declared.
- A *structure* block, in which fully interpreted predicates are enumerated, e.g. the bordering predicate in the map-colouring problem. This describes a (fixed) state of the world.
- A *theory* block, in which logical formulas are declared, e.g. the formula stating that countries that border ought to be coloured differently. These typically correspond to *constraints* that have to be satisfied.

2.4 Grounding

Before a FOL theory can be presented to a SAT solver⁹, it needs to be converted into a PL formula. This *grounding* of a formula boils down to eliminating the quantifiers over their (assumed to be finite) domain, i.e. converting the universal respectively existential quantifiers into what they are: a mere abbreviation for respectively conjunctions and disjunctions.¹⁰

As an example, take a formula $\forall x \in A : \exists y \in B : \forall z \in C : \varphi(x, y, z)$. This formula grounds to:

$$\bigwedge_{a_i \in A} \bigvee_{b_j \in B} \bigwedge_{c_k \in C} \varphi(a_i, b_j, c_k),$$

a formula in which the number of literals ranged over is equal to $|A| \cdot |B| \cdot |C|$. Broadly speaking, the aim of this thesis is to avoid generating these big conjunctions and disjunctions over their

⁵De Cat et al. [2018] introduce the *IDP system* (a reasoning engine), but also the *IDP language*. In this thesis, the term IDP refers to the latter use.

⁶Pronounced “FO dot”.

⁷FO[Int], FO[Real] and, in general, FO[Infinite].

⁸With file extension `.idp`.

⁹This is the case for all SAT solvers, specifically Z3 as well.

¹⁰We speak of a formula's *grounding* when we mean its equivalent PL formula after having eliminated all quantifiers. We speak of *the grounding* (in general) when we mean the set of groundings of all formulas in the theory.

full domains. The objective is not to build a better SAT solver as such, but to, as much as possible, (1) generate the equivalent grounded PL theory as efficiently as possible and (2) keep this grounding as small as possible, before it is handed to an out-of-the-box SAT solver.

2.5 SMTLIB

The Z3 solver requires us to present the grounding in the SMTLIB format. While the full syntax is described by Barrett et al. [2025], the following suffices for our purposes, and will look very familiar to the LISP user:

- There are atoms `true` and `false`.
- Predicates, functions and equality are written in a prefix notation, e.g. $P(x_0)$ would be written as `(P x0)` and $f(a_1, b_1) = c_1$ as `(= (f a1 b1) c1)`.
- Logical connectives are treated in a similar style to build nested expressions. Assuming `<e1>` and `<e2>` are expressions, negation is written as `(not <e1>)`. Conjunction and disjunction are respectively written as `(and <e1> <e2>)` and `(or <e1> <e2>)`, both with an arbitrary amount of arguments ≥ 1 .

2.6 RDBMSs

We briefly discuss some properties of the two Relational Database Management Systems (RDBMS) we will use in the implementation (refer to Chapter 3).

SQLite is the world’s most used database engine. It focuses on being lightweight, is self-contained and doesn’t depend on a server: it stores a database on-disk in a file. SQLite is *row-oriented*, meaning that tables are stored as a sequence of rows.

DuckDB focuses on efficiency. It has a strong query optimiser and has a *column-oriented* approach: tables are stored as a sequence of columns. This allows for faster processing of large chunks of data [Zuhra, 2023]. Although DuckDB supports storing and connecting to an on-disk database, one of its advantages is the possibility to connect to an in-memory (RAM) database. This eliminates file read/write operations and increases efficiency. A consequence is that the database only “lives” throughout the execution of the host process, in our case the Python system (refer to Chapter 4).

2.7 The Goal of the Thesis

As introduced in Section 2.4, all quantified formulas can in principle be grounded as an equivalent (combination of) big disjunctions and/or conjunctions, effectively ranging over the full Cartesian product of all sets involved. In plenty of cases, however, these equivalent formulas have a lot of subformulas which are trivially false or true, and these trivialities need to be eliminated if part

of a disjunction or conjunction.¹¹ This work of generating a huge formula which then needs to be simplified, yields an overall slow SAT solver.

The reason that the naively grounded formula can be simplified, is because it potentially contains subformulas the truth of which can be established based on facts listed in the *structure* block of an IDP file. Take, again, the classical map-colouring problem, in which countries that share a border have to be coloured differently:

$$\forall x, y \in \text{Country: } (\text{bordering}(x, y) \Rightarrow \text{colour_of_country}(x) \neq \text{colour_of_country}(y)).$$

Naively, if the set A represents the countries, P the bordering predicate and f the colouring function, we obtain for this formula φ the following, the size of which is $|A|^2$:

$$\varphi \Leftrightarrow \bigwedge_{x \in A} \bigwedge_{y \in A} P(x, y) \Rightarrow (f(x) \neq f(y)).$$

A lot of the implications conjuncted over, however, will, when countries *don't* border¹², reduce to $\top$, because in those cases the implication holds trivially. If we define S to be the set of all couples of bordering countries, i.e. exactly those constants (x, y) for which $P(x, y)$ holds¹³, we can restrict ourselves to this equivalence:

$$\varphi \Leftrightarrow \bigwedge_{(x, y) \in S} (f(x) \neq f(y)).$$

One could argue that this insight merely shifts the burden onto finding those couples (x, y) . The setup of this thesis research was inspired by Niu et al. [2011], who proposed a relational database to keep track of fully interpreted predicates. As will be explained in the next chapter, the proposed solution will build and use a database to efficiently *query* for those parts of quantified formulas that should be part of the grounded formula – S in the example above –, and explain which (and why) other parts of the full Cartesian product can be ignored – $(A \times A) \setminus S$ in the example above.

Rather than follow an approach of “generate-then-simplify”, we will investigate how, from the offset, having to generate the large grounding can be avoided. We will stick to the principle of being as concise as possible: the grounding will be *minimal*¹⁴ in the following sense. If the grounding contains subformulas which are function (in)equality, a predicate or predicate negation, i.e. one of the following:

- $f(\vec{x}_0) = y_0$,
- $f(\vec{x}_0) \neq y_0$,
- $p(\vec{x}_0)$,
- $\neg p(\vec{x}_0)$,

¹¹Through repeated simplifications: $(\perp \vee p) \Leftrightarrow p$, $(\top \vee p) \Leftrightarrow \top$, $(\top \wedge p) \Leftrightarrow p$ and $(\perp \wedge p) \Leftrightarrow \perp$.

¹²A situation which will occur for *most* pairs of countries on a realistic map, as Euler’s formula states that for planar graphs, $e \leq 3v - 6$, so the quadratic size of the conjunction can be reduced to a linear size!

¹³Also called its *satisfying set*.

¹⁴That the following holds will become clear in the next chapter, a formal proof of minimality in this sense is beyond the scope of this thesis.

this implies that the truth value of this subformula is *not* in any of the database tables (i.e. is not known to be true or false). Additionally, all $\top$ and $\perp$ atoms have been eliminated from conjunctions and disjunctions (e.g. p instead of $p \vee \perp$ etc.).

*I seemed to have lost all soul or sensation but
for this one pursuit.*

Mary Shelley

3

The IDP-Z3-DB System

In this chapter, we explain how the system that was implemented works and what the theoretical results are that are needed to prove its correctness. We concentrate on the high-level conceptual side of the system, more technical details about the implementation are postponed to the next chapter. The 4 sections below correspond to the 4 phases the system executes, in this order.

- 3.1 Parsing the IDP file and building a syntax tree.
- 3.2 Traversing (parts of) this tree to set up a database of types, fully interpreted predicates and some other known facts.
- 3.3 Using this database to ground all the formulas in the theory.
- 3.4 Presenting this grounding to the Z3 solver.

3.1 Parsing

The IDP file containing the full problem description is translated to a syntax tree, with – as explained in Section 2.3 – as main elements a *vocabulary*, *structure* and *theory* block. The former two will be used to set up the database, the theory block contains the formulas to ground, although some of them will be used to update the database as well (refer to Section 3.2.4).

Before we continue the explanation of how the system works, we slip in a digression on *inductive definitions*. De Cat et al. [2018] argue that one of the important additions of IDP to classical logic is support for inductive definitions. These are recursive definitions for predicates and functions, typically consisting of a recursive statement and a base case, but are not expressible in FOL.¹

¹Intuitively, because otherwise one could have a chain of arbitrary length.

One application is the notion of a transitive closure in graph theory. If one had a predicate E stating which vertices are connected by edges, an inductive definition for a predicate T expressing the transitive closure under E , could, in the language of FO[.], be defined with two rules:

```
{
  !x,y: T(x,y) <- E(x,y).
  !x,y: T(x,y) <- ?z: T(x,z) & (z,y).
}
```

Inductive definitions are not a part of FO[Core] and dealing with them in general would have led us to far. Some of the benchmarks of DIRT (see Section 5.1.1), however, make use of them, luckily not in an essential way, e.g. to define a triangle predicate p in a graph:

```
!x, y, z in Node: p(x, y, z) <- edge(x, y) & edge(y, z) & edge(x, z).
```

As shown by Denecker [2000], in case an inductive definition does not make use of recursion and consists of only one rule, it reduces to an ordinary FOL equivalence. While parsing², inductive definition blocks are therefore changed into an equivalence, but the user is warned that this only works in the naive case and yields an incorrect result in general.

3.2 The Database

3.2.1 Building the Database: Vocabulary

When parsing the *vocabulary* block of the IDP file, a database (further: *the database*) is built and manipulated, with either SQLite or DuckDB³ as RDBMS. For SQLite the database is on-disk, for DuckDB in-memory.

Types are defined as tables with as PK their name. E.g. `type A := { a1, a2, a3, a4 }` first yields a query to create the table:

```
CREATE TABLE "A" ("A" TEXT, PRIMARY KEY("A"))
```

and one to fill it with the appropriate values:

```
INSERT INTO A VALUES ("a1"), ("a2"), ("a3"), ("a4")
```

Exactly the same is generated in case types are only “filled” with constants in the *structure* block. Integer ranges are supported as well.

²This is why we slip in this paragraph in this section, even if dealing with inductive definitions in general is not a parsing problem, but we circumvent the problem in the parsing stage.

³In the earlier versions, we only used SQLite. Stijn Vansummen suggested to use this RDBMS, which sped up some queries but made others slower, refer to Chapter 5.

Predicates and **functions** are defined as association tables with FKs to the tables of their domain types and (in case of functions) their range type. Additionally, a column **truth** is defined which will hold the truth value of this particular assignment. Let us first have a look at the query to create the table of a predicate $P: A * A * B \rightarrow \text{Bool}$. Note that because of the double occurrence of the type A , aliases have to be defined.

```
CREATE TABLE "P" (
  A_1 TEXT, A_2 TEXT, B TEXT, truth INTEGER,
  PRIMARY KEY (A_1 , A_2 , B ),
  FOREIGN KEY ("A_1") REFERENCES "A"("A"),
  FOREIGN KEY ("A_2") REFERENCES "A"("A"),
  FOREIGN KEY ("B") REFERENCES "B"("B")
)
```

Similarly, for a function $f: A \rightarrow B$, we obtain:

```
CREATE TABLE "f" (
  A TEXT , B TEXT , truth INTEGER,
  PRIMARY KEY (A , B ),
  FOREIGN KEY ("A") REFERENCES "A"("A"),
  FOREIGN KEY ("B") REFERENCES "B"("B")
)
```

One could argue that for a function, only the domain should be the PK. For the example above – but the argument works in general – we could have opted to indeed take (**A**) as the PK, and treat the fact whether the column **B** has a constant from B , as indicating whether the function value is known or not. The decision to take (**A,B**) as PK and still store a **truth** value, has the following rationale. Say we have a formula in the theory stating that $\forall x \in A : f(x) \neq b_1$. While we still don't know the function values, at least we know *what they are not*, and inserting (as will be explained in Section 3.2.4) these tuples with a value 0 in the **truth** column, allows us to use this formula to make the database more precise and simplify formulas while grounding. This at the expense of increasing the size of the table.⁴

3.2.2 Building the Database: Structure

While all predicates start off their lives as partially interpreted, some turn out to be **fully interpreted** when parsing the *structure* block of the IDP file. For those that are, we drop the column **truth** and insert⁵ the values into the table. Should we have, e.g., for the predicate P introduced earlier in Section 3.2.1,

$$P := \{(a_1, a_1, b_1), (a_2, a_2, b_1), (a_3, a_3, b_1)\}.$$

the queries are

⁴This is an instance of the “Law of Conservation of Misery”.

⁵The **OR IGNORE** statement in the query below is a small technicality to circumvent potential repetition in the enumeration.

```

ALTER TABLE P DROP COLUMN truth
INSERT OR IGNORE INTO P VALUES
  ("a1", "a1", "b1"),
  ("a2", "a2", "b1"),
  ("a3", "a3", "b1")

```

Note that from now on, for a fully interpreted predicate, whether or not it holds for a certain tuple of domain elements, is equivalent to whether or not that particular tuple of elements occurs in the table.

For predicates that remain **partially interpreted** and **functions**⁶, the tables stay as they are. As stated in Chapter 1, for partially interpreted predicates and functions, a tuple of constants not occurring in the table means that we do not know (yet) whether the predicate holds on these, or whether the function has this particular value. Only when we know a certain truth value, will this table be populated with constants, which will be the topic of Section 3.2.4. First, we explain how the database can be consulted to find tuples satisfying a certain formula involving only fully interpreted predicates.

3.2.3 Consulting the Database: Finding Satisfying Sets

In the rest of this chapter, we will often have to determine the *satisfying set* (or *SAT-set*) S of a quantified formula $\varphi(\vec{x})$, where φ only contains fully interpreted predicates. Finding the set S can be done entirely by consulting the database, as φ contains only fully interpreted predicates. This is done by translating the logical connectives of φ in an SQL query, the correctness of which hinges on the fact that, as discussed in Section 3.2.2, a fully interpreted predicate holds on the constants (and only the constants) recorded in its database table.

In general, this query looks like this:

```
SELECT attributes FROM tables WHERE conditions
```

with:

- **attributes** listing all the PKs of the types occurring as free variables in φ , potentially using their alias (see next point);
- **tables** listing the tables of all the types occurring as free variables in φ , giving the same type an alias should it occur multiple times;
- **conditions** translating the subformulas of φ into a correct **WHERE** clause. This is done by a translator function $T(\varphi)$ which is defined recursively on the structure of φ ⁷:

$$- \text{ If } \varphi \Leftrightarrow \neg\psi, T(\varphi) = (\text{ NOT } T(\psi))$$

⁶One could also consider *fully interpreted functions*, but we chose not to. The reason is that they are equivalent to fully interpreted predicates: a fully interpreted function with, amongst others, e.g. $f(a_1) = b_1$, could be listed as $P(a_1, b_1)$ in the structure, and one could manually make P represent a function by ensuring that a_1 occurs in no other tuple satisfying P (besides together with b_1).

⁷Assuming equivalences have been converted to a conjunction of implications and implications to disjunctions.

- If $\varphi \Leftrightarrow (\psi_1 \wedge \psi_2)$, $T(\varphi) = (T(\psi_1) \text{ AND } T(\psi_2))$
- If $\varphi \Leftrightarrow (\psi_1 \vee \psi_2)$, $T(\varphi) = (T(\psi_1) \text{ OR } T(\psi_2))$
- If φ is an equality, say $x = y$, with corresponding PK attributes $X.X$ and $Y.Y$, $T(\varphi) = (X.X = Y.Y)$
- If φ is a predicate, $T(\varphi) = (\text{attributes IN predicate_selection_query})$, in which
 - * **attributes** is the list of free variable attributes occurring in φ
 - * **predicate_selection_query** a query selecting the variable attributes from the predicate table, with an optional **WHERE** clause to constrain constants.

We will illustrate this by means of two examples. Firstly, say $\varphi(x, y)$ is $\neg P(x) \wedge Q(x, y)$, for $x \in A$ and $y \in B$. Finding its satisfying set S is achieved by the query

```
SELECT A.A, B.B FROM A A, B B WHERE (
  (NOT ((A.A) IN (SELECT P.A FROM P)))          -- ¬P(x)
  AND ((A.A,B.B) IN (SELECT Q.A, Q.B FROM Q))  -- ^ Q(x,y)
)
```

Secondly, say $\varphi(x, y)$ is $R(x, y, a_1) \vee (x = y)$, for $x, y \in A$. Its satisfying set is found as the result of the following query, which illustrates both the use of aliases (necessary as A occurs multiple times) and the constraint on the constant a_1 .

```
SELECT A_0.A, A_1.A FROM A A_0, A A_1 WHERE (
  ((A_0.A,A_1.A) IN
    (SELECT R.A_0, R.A_1 FROM R WHERE R.A_2 = "a1")) -- R(x,y,a1)
  OR (A_0.A = A_1.A)                                -- v (x=y)
)
```

3.2.4 Updating the Database: Fact Queries

While processing formulas in the *theory* block is mainly discussed in Section 3.3, certain formulas in the theory block should be used first to update the database, and generate what we will refer to as *fact queries*.⁸

Unquantified Formulas

Firstly, we'll consider two kinds of unquantified formulas. They are formulas that state (1) whether certain partially interpreted predicates hold on certain constants, or (2) what the image of a certain function applied to constants is (or is not).

For the first kind, say we have a predicate $q \subseteq A \times B$ and a formula $\sim q(a_2, b_1)$. In this case, the tuple is inserted into the table for **q** and the corresponding **truth** column is set to 0 (1 in case the negation wouldn't have been there):

⁸Which are always **INSERT INTO** queries.

```
INSERT INTO q VALUES
  ("a2", "b1", 0)
```

For the second kind, in case e.g. $f(a_1) \neq b_1$, the query is similar:

```
INSERT INTO f VALUES
  ("a1", "b1", 0)
```

Note that in case we knew $f(a_1) = b_1$, there would again have been a 1 instead of 0 as a truth value, but additionally, the system ensures that when grounding, a statement like $f(a_1) = b$ is evaluated to $\perp$ for all $b \neq b_1$.

Quantified Formulas

There are, however, also two⁹ kinds of *quantified* formulas that give rise to a fact query as well.

For the first kind, say that $\chi(\vec{x})$ is a formula (or a conjunction of subformulas) that is (or are) either:

- stating whether a partially interpreted predicate holds;
- the negation of such a statement;
- stating whether a function value is equal to a constant;
- the negation of such a statement.

Then $\forall \vec{x} : \chi(\vec{x})$ can give rise to fact queries on all the conjunction elements of χ , over the full Cartesian product of the types involved in $\vec{x}$. As an example, consider

$$\forall x, y \in A : ((r(x, y, b_3) \wedge f(y) \neq b_2).$$

This formula gives rise to the following fact queries:

```
INSERT OR IGNORE INTO r (A_1,A_2,B,truth) VALUES
  ("a1","a1","b3",1),
  ("a1","a2","b3",1),
  ("a1","a3","b3",1),
  ("a1","a4","b3",1),
  [enumeration continues over all of AxA for first two constants]
```

⁹There are many more when one thinks about it, but treating them all exhaustively would needlessly complicate matters.

```

INSERT OR IGNORE INTO f (A,B,truth) VALUES
  ("a1","b2",0),
  ("a2","b2",0),
  ("a3","b2",0),
  ("a4","b2",0),
  [enumeration continues over all of A for first constant]

```

For the second kind, say that $\chi(\vec{x})$ is as above, and $\varphi(\vec{x})$ is a formula with only fully interpreted subformulas. Then $\forall \vec{x} : \varphi(\vec{x}) \Rightarrow \chi(\vec{x})$ can give rise to similar fact queries, but now only ranging over those tuples in S , with S the satisfying set of $\varphi(\vec{x})$, obtained through querying the database, as explained in Section 3.2.3. As an example, consider

$$\forall x, y \in A : (P(x) \wedge Q(y, b_1)) \Rightarrow ((r(x, y, b_3) \wedge f(y) \neq b_2),$$

and assume only $P(a_1)$, $P(a_2)$ and $Q(a_1, b_1)$ hold. This formula yields the following two fact queries:

```

INSERT OR IGNORE INTO r (A_1,A_2,B,truth) VALUES
  ("a1","a1","b3",1),
  ("a2","a1","b3",1)

```

```

INSERT OR IGNORE INTO f (A,B,truth) VALUES
  ("a1","b2",0),
  ("a1","b2",0)

```

Fact queries assert truth values and are generated by a first pass over all formulas in the *theory* block. Formulas that have given rise to a fact query are flagged as “used” and will not be grounded. How to process more complex formulas is the topic of the next section.

3.3 Grounding of Formulas

In the theory block of the IDP file, we find formulas that need to be grounded, unless they have been used in a fact query, as explained in Section 3.2.4.

To compare (refer to Section 5.2) the more advanced grounding methods that will be developed in this section, the naive version of grounding has been implemented as well, which translates existential and universal quantifiers into respectively disjunctions and conjunctions over the domains involved (refer to Section 2.4) and does not make use of the database to simplify the grounding. Instead, it lists fully interpreted predicates in the grounding as well, based on the information in the database.¹⁰

The added value of using a database in grounding a formula, is revealed when grounding quantified formulas, because, as stated in Section 2.7, significant parts of the large conjunctions and disjunctions can be avoided by consulting the database.

¹⁰That means that even if we ground naively, the database is still used, but only to write the predicates to the file containing the SMT grounding.

The aim of the following sections is to explain how the grounding of different kinds of formulas in the theory can be computed using the database that has been built in the previous phase (refer to Section 3.2).

3.3.1 Formulas without Quantifiers

Formulas with no quantifiers are straightforward to ground. Their logical connectives are translated to the SMTLIB syntax, and their atoms are simplified if possible:

- Fully interpreted predicates are queried for in the database and therefore simply yield $\top$ or $\perp$.
- Partially interpreted predicates are queried for and yield $\top$ or $\perp$ in case their truth value has been determined earlier by a fact query.
- Equality statements on functions of the form $f(\vec{x}_0) = c$ can be resolved to $\top$ or $\perp$ in case the function value is known.
- By querying for a **truth** value of 0, statements of the form $f(\vec{x}_0) \neq c$ can be resolved to $\top$ or $\perp$ as well.
- Other statements involving functions are evaluated to their function value, in case that is known.
- In all other cases, atoms are merely translated to SMT syntax.
- Simplification rules are applied on the fly.

By means of an example, consider a formula $f(x_0) = g(y_0, z_0)$, involving functions, in case we know that $g(y_0, z_0) = w_0$ (processed with a fact query). This formula grounds to $(= (\mathbf{f} \ \mathbf{x0}) \ \mathbf{w0})$. Similarly, assuming $P(x_0)$ is known to not hold, $P(x_0) \vee p(x_0)$ grounds to $(\mathbf{p} \ \mathbf{x0})$.

This reveals a limitation for fact queries. In the first example above, now that we know that $f(x_0) = w_0$, we could update this information in the database with a fact query. The second example, $P(x_0) \vee p(x_0)$, is grounded in the current phase, but given that we know $P(x_0)$ does *not* hold, the formula reduces to $p(x_0)$ and could therefore also have been giving rise to a fact query, having the benefit of making a truth value in the table for p more precise. Implementing this reasoning in general, however, would involve a kind of transitive closure approach. The overhead thus created was deemed not worth the effort, given its limited added value.

3.3.2 Formulas with Only Universal Quantifiers

In this and the following sections, unless explicitly mentioned, we assume that all quantifiers involved have been explicitly written in the formulas that are mentioned in the text, and that subformulas “deeper” are quantifier-free. We return to the general case, where this does not necessarily hold, in Section 3.3.7. E.g. in the formula below, we assume that neither φ nor χ contain quantifiers.

First, let's consider a universally quantified formula

$$\forall \vec{x} : \varphi(\vec{x}) \Rightarrow \chi(\vec{x}),$$

with the assumption that the subformulas of φ contain only fully interpreted predicates, i.e. predicates mentioned in the *structure*.¹¹ Because of the structure of formulas like these, if we know that the set of tuples $S := \{\vec{a}_1, \dots, \vec{a}_n\}$ is the satisfying set of φ , this formula can be grounded to $\bigwedge_{\vec{a}_i \in S} \chi(\vec{a}_i)$. How to find S has been described higher, in Section 3.2.3.

If a universally quantified formula doesn't satisfy the format above, it can be converted to an equivalent conjunction of formulas that do, using the following procedure, inspired by [Van Laer et al., 2024]. Consider $\psi \Leftrightarrow \forall \vec{x} : \varphi(\vec{x})$, with φ having n fully interpreted predicates P_i used in its subformulas.

Let $P' := \{\bigwedge_{i=1}^n (-)P_i\}$ be the set which contains all $N := 2^n$ possibilities of in- or excluding “ $\neg$ ”. We write φ'_j for the formula obtained from φ by substituting $\top$ and $\perp$ for respectively occurrences of predicates P_i or $\neg P_i$ in a formula $P'_j \in P'$. Note that the formulas P'_j are exhaustive and mutually exclusive. Also $\bigvee_{j=1}^N \varphi'_j \Leftrightarrow \varphi$, because all possibilities of occurrences of $(-)P_i$ in them are covered. These facts combined ensure that $\varphi \Leftrightarrow \bigwedge_{j=1}^N (P'_j \Rightarrow \varphi'_j)$.

Note that the correctness of the approach crucially hinges on the universal quantifier $\forall$ commuting with conjunction $\wedge$ ¹²:

$$\begin{aligned} \psi &\Leftrightarrow \forall \vec{x} : \varphi(\vec{x}) \\ &\Leftrightarrow \forall \vec{x} : \bigwedge_{j=1}^N (P'_j(\vec{x}) \Rightarrow \varphi'_j(\vec{x})) \\ &\Leftrightarrow \bigwedge_{j=1}^N \forall \vec{x} : (P'_j(\vec{x}) \Rightarrow \varphi'_j(\vec{x})) \\ &\Leftrightarrow \bigwedge_{j=1}^N \bigwedge_{\vec{x}_i \in X} (P'_j(\vec{x}_i) \Rightarrow \varphi'_j(\vec{x}_i)). \end{aligned}$$

The inner conjunction of the last equivalence reduces to ranging over only those $\vec{x}_i$ in the SAT-set of P'_j (as for other tuples, the implication holds trivially).

Let's illustrate this procedure on grounding $\forall x, y : \varphi(x, y)$ for φ :

$$\neg P(x) \vee (Q(x, y) \Rightarrow (p(x, y) \wedge f(x) = y)).$$

Applying the procedure above on the 2 fully interpreted predicates P and Q , yields that φ is equivalent to $\bigwedge_{j=1}^4 \varphi_j$, for φ_j :

$$\begin{aligned} \varphi_1 : & (P(x) \wedge Q(x, y)) \Rightarrow (\neg \top \vee (\top \Rightarrow (p(x, y) \wedge f(x) = y))) \\ \varphi_2 : & (P(x) \wedge \neg Q(x, y)) \Rightarrow (\neg \top \vee (\perp \Rightarrow (p(x, y) \wedge f(x) = y))) \\ \varphi_3 : & (\neg P(x) \wedge Q(x, y)) \Rightarrow (\neg \perp \vee (\top \Rightarrow (p(x, y) \wedge f(x) = y))) \\ \varphi_4 : & (\neg P(x) \wedge \neg Q(x, y)) \Rightarrow (\neg \perp \vee (\perp \Rightarrow (p(x, y) \wedge f(x) = y))) \end{aligned}$$

¹¹The map-colouring constraint used as an example in Section 2.7, satisfies this format.

¹²A fact we get back to in Section 3.3.4.

Note that the consequents in φ_2 , φ_3 and φ_4 all simplify to $\top$, so these do not contribute to the grounding at all. Furthermore, the consequent of φ_1 simplifies to $p(x, y) \wedge f(x) = y$. Once the satisfying set S of $P(x) \wedge Q(x, y)$ has been read from the database, the original formula $\forall x, y : \varphi(x, y)$ grounds to

$$\bigwedge_{(x_i, y_i) \in S} p(x_i, y_i) \wedge f(x_i) = y_i.$$

The fact that in this example all of the φ_i simplified significantly, in some cases even to a trivial formula, is generic. Simplifying the consequent of the implications before finding the satisfying set of its antecedent, has been implemented and avoids a lot of redundant work, even if the worst-case scenario yields 2^n subformulas, with n the number of fully interpreted predicates in the given formula.

In general, if the consequent φ'_j simplifies to $\top$, the implication is ignored in grounding the formula, because an implication with a true consequent holds trivially. If the consequent simplifies to $\perp$, a check is performed whether the antecedent P'_j holds on certain tuples of constants (i.e. checking whether the database contains entries satisfying the antecedent). If so, the entire universally quantified formula is immediately grounded to $\perp$.¹³ Only in case the consequent *doesn't* simplify (entirely), like φ_1 in the example above, “actual” grounding work is to be done based on conjuncting the consequent over the satisfying set of the antecedent (found by consulting the database, as described earlier).

We also note that when calculating the grounding, simplification is performed and the database is queried for known facts. Take again the formula from above and its grounding:

$$\forall x, y : \varphi(x, y) \quad \Leftrightarrow \quad \bigwedge_{(x_i, y_i) \in S} p(x_i, y_i) \wedge f(x_i) = y_i.$$

Say we find, for $(x_0, y_0) \in S$, after querying the database, that it had been established with a fact query that $f(x_0) = y_0$. Then this element of the conjunction is listed as $p(x_0, y_0)$ (rather than $p(x_0, y_0) \wedge \top$). Say, on the other hand, that we find that $\neg p(x_0, y_0)$ holds, then building the conjunction is halted and the theory is immediately established as UNSAT. These simplifications are performed as well when grounding the other quantifier types discussed below, sometimes drastically reducing grounding and solving times (refer to Chapter 5).

3.3.3 Formulas with Only Existential Quantifiers

Like in Section 3.3.2, we will first explain how to ground formulas that have a certain standard form, and then show how to convert other formulas to equivalent ones in this standard form.

Here, the standard form is this one:

$$\exists \vec{x} : \varphi(\vec{x}) \wedge \chi(\vec{x}),$$

again with the assumptions that neither φ nor χ contain quantifiers and that the subformulas of φ contain only fully interpreted predicates. In this case, if the set $S := \{\vec{x}_1, \dots, \vec{x}_n\}$ is the satisfying set of φ , this formula can be grounded to $\bigvee_{\vec{x}_i \in S} \chi(\vec{x}_i)$. As earlier, S can be obtained by querying the database.

¹³For this reason, these cases are considered *first*, to flag an UNSAT situation as fast as possible.

If an existentially quantified formula doesn't satisfy the format above, it can be converted to an equivalent disjunction of formulas, with a procedure similar to the one described in Section 3.3.2, with the only difference that in this case, $\varphi \Leftrightarrow \bigvee_{j=1}^N (P'_j \wedge \varphi'_j)$.

Again, the correctness is guaranteed by commuting the quantifier and the connective:

$$\begin{aligned}
\psi &\Leftrightarrow \exists \vec{x} : \varphi(\vec{x}) \\
&\Leftrightarrow \exists \vec{x} : \bigvee_{j=1}^N (P'_j(\vec{x}) \wedge \varphi'_j(\vec{x})) \\
&\Leftrightarrow \bigvee_{j=1}^N \exists \vec{x} : (P'_j(\vec{x}) \wedge \varphi'_j(\vec{x})) \\
&\Leftrightarrow \bigvee_{j=1}^N \bigvee_{\vec{x}_i \in X} (P'_j(\vec{x}_i) \wedge \varphi'_j(\vec{x}_i)).
\end{aligned}$$

The inner disjunction of the last equivalence reduces to ranging over only the SAT-set of P'_j (as for other tuples, the conjunction is trivially false and disappears from the disjunction).

The simplification rules work slightly differently as well. Now, if a certain $\varphi'_j \Leftrightarrow \perp$, we ignore it in the grounding. If $\varphi'_j \Leftrightarrow \top$, a check is performed whether the corresponding left-hand P'_j of the conjunction holds for at least some combination of constants, i.e. whether the result set of the query for its SAT-set is non-empty. If this is indeed the case, the entire formula is immediately grounded to $\top$.¹⁴ If the formula doesn't simplify to a trivial one, we disjunct the right-hand side φ'_j over the satisfying set of the left-hand side P'_j .

3.3.4 Formulas with Mixed Quantifiers: $\forall \exists$

Grounding a formula of e.g. the form $\forall x \in A : P(x) \Rightarrow p(x)$ in the way we did in Section 3.3.2 was efficient and avoided the conjunction ranging over the entirety of A . The reason for this was that we avoided $x \in A$ where the antecedent didn't hold, which would have made the implication trivially true and would only have contributed a " $\wedge \top$ " in the conjunction.

We will follow a similar strategy when grounding a formula of the form $\forall \vec{x} : \exists \vec{y} : \varphi(\vec{x}, \vec{y})$, where φ is a quantifier-free formula containing any combination of function equalities, partially and fully interpreted predicates. In this case, because

$$\forall \vec{x} : \exists \vec{y} : \varphi(\vec{x}, \vec{y}) \Leftrightarrow \bigwedge_{\vec{x}_i} \bigvee_{\vec{y}_j} \varphi(\vec{x}_i, \vec{y}_j),$$

any $\varphi(\vec{x}_i, \vec{y}_j)$ that is trivially true, makes the inner disjunction true, and therefore again only contributes a " $\wedge \top$ " in the outer conjunction and should not be part of the grounding. It is therefore only relevant to consider those constants which *don't* make φ trivially true. This will be done in a way that is similar to the approach in Section 3.3.2: considering all possible combinations of facts we can query for in the database.

To formalise this, we need some more notations and conventions. First of all, we will assume that there is only one universally and one existentially quantified variable, as this allows us to omit

¹⁴Again, these cases are considered first.

the vector arrows in notation. This is only to increase readability, the general case is completely analogous. With this convention, the formula that needs grounding is

$$\psi \Leftrightarrow \forall x \in X : \exists y \in Y : \varphi(x, y).$$

Say that φ contains n fully interpreted predicates P_i . Let, like earlier, $P' := \{\bigwedge_{i=1}^n (\neg)P_i\}$, which contains all $N := 2^n$ possibilities of in- or excluding “ $\neg$ ”. We write φ'_j for the formula obtained from φ by substituting $\top$ and $\perp$ for respectively occurrences of predicates P_i or $\neg P_i$ in a formula $P'_j \in P'$. Additionally, we define S_j as the set of tuples in $X \times Y$ that satisfy the formula $P'_j \in P'$. The SAT-sets S_j can be computed by querying the database. Note that $\bigcup_{j=1}^N S_j = X \times Y$ and that the sets S_j are mutually disjoint.

Using this partition, we have the following equivalences. Note that, like in 3.3.2 and 3.3.3, the commuting of the quantifier and the connective, is essential.

$$\begin{aligned} \psi &\Leftrightarrow \forall x \in X : \exists y \in Y : \bigvee_{j=1}^N ((x, y) \in S_j \wedge \varphi(x, y)) \\ &\Leftrightarrow \forall x \in X : \bigvee_{j=1}^N \exists y \in Y : ((x, y) \in S_j \wedge \varphi(x, y)) \\ &\Leftrightarrow \forall x \in X : \bigvee_{j=1}^N \exists y \in Y : ((x, y) \in S_j \wedge \varphi'_j(x, y)). \end{aligned}$$

This reveals how we can avoid those S_j where φ is trivially true. If for a certain $x \in X$, there is a j for which its corresponding φ'_j is equivalent to $\top$ and there is at least a tuple (x, y) in S_j , the whole inner disjunction collapses to $\top$ and no grounding work is to be done for that x . This justifies the following definitions.

Let $S_\top := \bigcup\{S_j \mid \varphi'_j \Leftrightarrow \top\}$ and $X_\top := \{x \in X \mid \exists y \in Y \wedge (x, y) \in S_\top\}$. Then we can define X_{NT} (for “non-trivial”) as $X_{NT} := X \setminus X_\top$. Before we plug this into the equivalence above, however, we also note that for certain j , φ could turn out to be equivalent to $\perp$, which can also be avoided in the disjunction. We therefore define

$$J_{NT} := \{j \in \{1, \dots, N\} \mid \varphi'_j \not\Leftrightarrow \top \wedge \varphi'_j \not\Leftrightarrow \perp\},$$

where trivial formulas are avoided, intuitively speaking, for different reasons: trivially true because we will already have avoided those tuples satisfying them anyway; trivially false because they do not contribute in the disjunction (and the formula will have to be made true by other j 's).

We also want a notation to express the set of elements of Y that occur with a certain x in an S_j . This we will write as

$$S_j^x := \{y \in Y \mid (x, y) \in S_j\}.$$

With these notations we are ready to fully simplify the equivalence of ψ :

$$\begin{aligned}
\psi &\Leftrightarrow \forall x \in X_{NT} : \bigvee_{j=1}^N \exists y \in Y : ((x, y) \in S_j \wedge \varphi'_j(x, y)) \\
&\Leftrightarrow \forall x \in X_{NT} : \bigvee_{j \in J_{NT}} \exists y \in Y : ((x, y) \in S_j \wedge \varphi'_j(x, y)) \\
&\Leftrightarrow \bigwedge_{x \in X_{NT}} \bigvee_{j \in J_{NT}} \bigvee_{y \in S_j^x} \varphi'_j(x, y).
\end{aligned}$$

We note two interesting edge-cases:

- Should X_{NT} be empty, i.e. we can find a trivial fulfillment for *all* x , then $\psi \Leftrightarrow \top$.
- Should X_{NT} be non-empty but the inner disjunctions vanish, either because J_{NT} is empty or all S_j^x are empty for a certain x , then $\psi \Leftrightarrow \perp$ (and the problem UNSAT).

We will illustrate this on the following example:

$$\psi \Leftrightarrow \forall x \in A : \exists y \in B : (P(x) \Rightarrow p(x, y)) \wedge Q(x, y).$$

We find the following:

$$\begin{aligned}
P'_1 &= P(x) \wedge Q(x, y) : \varphi'_1 \Leftrightarrow p(x, y) \\
P'_2 &= P(x) \wedge \neg Q(x, y) : \varphi'_2 \Leftrightarrow \perp \\
P'_3 &= \neg P(x) \wedge Q(x, y) : \varphi'_3 \Leftrightarrow \top \\
P'_4 &= \neg P(x) \wedge \neg Q(x, y) : \varphi'_4 \Leftrightarrow \perp
\end{aligned}$$

In this case, $J_{NT} = \{1\}$. For what follows, let's assume both A and B contain only two constants.

Firstly, say we know that (only) $P(a_2)$ and $Q(a_1, b_1)$ hold, clearly an UNSAT configuration, as no $Q(a_2, y)$ can be found. Then $A_\top = \{a_1\}$ and $A_{NT} = \{a_2\}$. In this case, the outer conjunction only ranges over this singleton, but also $S_1 = \emptyset$, so we encounter the second edge-case: $\psi \Leftrightarrow \perp$.

Secondly, let's assume additionally $Q(a_2, b_2)$ holds. $A_\top$ and A_{NT} remain the same but now $S_1 = \{(a_2, b_2)\}$. In this case, ψ is equivalent to $p(a_2, b_2)$, which corresponds to the intuition: now that the part of the formula involving Q is “fixed” by the extra assumption, and the implication still holds trivially for a_1 because $\neg P(a_1)$, $p(a_2, b_2)$ needs to hold.

Thirdly, let's say *only* $Q(a_1, b_1)$ and $Q(a_2, b_2)$ hold, so P is never true. In this case, $A_\top = \{a_1, a_2\}$ and $A_{NT} = \emptyset$, so indeed $\psi \Leftrightarrow \top$, i.e. the first edge-case.

3.3.5 Formulas with Mixed Quantifiers: $\exists \forall$

The reasoning of Section 3.3.4 can be repeated based on the insights of Section 3.3.3, when dealing with formulas $\exists \vec{x} : \forall \vec{y} : \varphi(\vec{x}, \vec{y})$. When grounding $\exists x \in A : P(x) \wedge p(x)$, we could

restrict ourselves to a disjunction over those $x \in A$ for which $P(x)$ held, as other constants only contributed a trivial “ $\vee \perp$ ”.

Now, the formula in question is equivalent to a disjunction of conjunctions:

$$\exists \vec{x} : \forall \vec{y} : \varphi(\vec{x}, \vec{y}) \Leftrightarrow \bigvee_{\vec{x}_i} \bigwedge_{\vec{y}_j} \varphi(\vec{x}_i, \vec{y}_j).$$

Therefore, any $\varphi(\vec{x}_i, \vec{y}_j)$ that is trivially false, makes the inner conjunction false, and therefore again only contributes a “ $\vee \perp$ ” in the outer disjunction and should not be part of the grounding. It is therefore only relevant to consider those constants which *don't* make φ trivially false. Where the analogy with the previous section breaks down, is that by partitioning, φ turned out to be equivalent to a *disjunction*, which does not commute with the now inner universal quantifier.

With the same notation and conventions as in Section 3.3.4, the formula to ground is

$$\psi \Leftrightarrow \exists x \in X : \forall y \in Y : \varphi(x, y).$$

We now define $S_\perp := \bigcup \{S_j \in P' \mid \varphi'_j \Leftrightarrow \perp\}$ and $X_\perp := \{x \in X \mid \exists y \in Y \wedge (x, y) \in S_\perp\}$. In other words, $X_\perp$ are those x 's that have at least one counterexample, and will never be good candidates as the witness of the existential quantifier. For this case, we define $X_{NT} := X \setminus X_\perp$. In the inner conjunction, we can avoid those $y \in Y$ that make φ trivially true. For those, we define, for a given $x \in X$:

$$T_Y(x) := \{y \in Y \mid \exists j (\varphi'_j \Leftrightarrow \top \wedge (x, y) \in S_j)\}.$$

We will write $NT_Y(x)$ for $Y \setminus T_Y(x)$. Now, ψ can be rewritten as follows:

$$\begin{aligned} \psi &\Leftrightarrow \exists x \in X_{NT} : \forall y \in Y : \varphi(x, y) \\ &\Leftrightarrow \exists x \in X_{NT} : \forall y \in NT_Y(x) : \varphi(x, y) \\ &\Leftrightarrow \bigvee_{x \in X_{NT}} \bigwedge_{y \in NT_Y(x)} \varphi(x, y). \end{aligned}$$

Note that, unlike in the previous paragraphs, the simplified φ'_j cannot be used just like that. Technically, one could fetch the S_j of which the pair (x, y) is the element (and then its corresponding φ'_j), but that would create unwanted overhead. The formula will, however, simplify “naturally”, as fully interpreted predicates will evaluate to either $\top$ or $\perp$ by consulting the database (this is illustrated in the example below). This – the fact that simplifications have to be done “on the fly” – and the $NT_Y(x)$ “projection” sets which have to be established, make this by far the more complicated quantifier succession (in comparison to Sections 3.3.2, 3.3.3 and 3.3.4), which will show in Section 5.2.2, especially case (A8Bf).

There are again two edge-cases to be considered, the mirror image of the ones of the previous section:

- Should X_{NT} be empty, i.e. there are no “candidates” for witnesses x to begin with, then $\psi \Leftrightarrow \perp$ (and the problem UNSAT).
- Should X_{NT} be non-empty, but for a certain $x \in X_{NT}$ all the inner conjunctions vanish because $NT_Y(x)$ is empty, then $\psi \Leftrightarrow \top$.

We will illustrate this on the example

$$\psi \Leftrightarrow \exists x \in A : \forall y \in B : (P(x) \Rightarrow p(x, y)) \wedge Q(x, y),$$

which has the same set P' as the example of the previous section. We again assume both A and B have two constants.

Firstly, say that only $Q(a_1, b_1)$ and $Q(a_1, b_2)$ hold. $A_\perp = \{a_2\}$ and $A_{NT} = \{a_1\}$. As $S_3 = \{(a_1, b_1), (a_1, b_2)\}$, $T_B(a_1) = \{b_1, b_2\}$ and $NT_B(a_1) = \emptyset$. This means we encountered the second edge-case, as indeed a_1 is the witness in question, and $\psi \Leftrightarrow \top$.

Secondly, let's assume that additionally, $P(a_1)$ holds. Now, while $A_\perp$ and A_{NT} remain the same, $S_3 = \emptyset = T_B(a_1)$. This means that

$$\psi \Leftrightarrow ((P(a_1) \Rightarrow p(a_1, b_1)) \wedge Q(a_1, b_1)) \wedge ((P(a_1) \Rightarrow p(a_1, b_2)) \wedge Q(a_1, b_2)),$$

which will simplify to $p(a_1, b_1) \wedge p(a_1, b_2)$, corresponding to the intuition that, as a_1 is the only “good” candidate as a witness, we have to ensure that the implication holds for all b 's.

Thirdly, say only $Q(a_1, b_1)$ and $Q(a_2, b_2)$ hold, clearly an UNSAT situation. Then indeed $A_\perp = \{a_1, b_2\}$ and $A_{NT} = \emptyset$, so the first edge-case is encountered and $\psi \Leftrightarrow \perp$.

3.3.6 Formulas with Mixed Quantifiers: In General

The approach of the previous two Sections, 3.3.4 and 3.3.5, cannot be generalised in case of more (mixed) combinations of quantifiers.

If one wants to avoid “unravelling” a formula like the one presented in 2.4 into a tree-like mix of conjunctions and disjunctions, leaving simplification to the solver, the given formula can be transformed into a conjunction of equivalent ones, which does allow to ground them with the procedures covered above. The price to pay is that we have to introduce new symbols and formulas.

Consider the formula

$$\psi_1 \Leftrightarrow \exists x : \forall y : \exists z : \varphi(x, y, z).$$

We will introduce the following predicate p_φ , with x and y free variables:

$$p_\varphi(x, y) \Leftrightarrow \exists z : \varphi(x, y, z).$$

Then we can substitute the original ψ_1 in the theory by the equivalent formula $\exists x : \forall y : p_\varphi(x, y)$, which can be grounded using the techniques described earlier. We also add the following formula to the theory, to properly define p_φ :

$$\begin{aligned} & \forall x : \forall y : (p_\varphi(x, y) \Leftrightarrow \exists z : \varphi(x, y, z)) \\ \Leftrightarrow & \forall x : \forall y : ((p_\varphi(x, y) \Rightarrow \exists z : \varphi(x, y, z)) \wedge (p_\varphi(x, y) \Leftarrow \exists z : \varphi(x, y, z))) \\ \Leftrightarrow & \forall x : \forall y : (p_\varphi(x, y) \Rightarrow \exists z : \varphi(x, y, z)) \wedge \forall x : \forall y : (p_\varphi(x, y) \Leftarrow \exists z : \varphi(x, y, z)) \\ \Leftrightarrow & (\forall x : \forall y : \exists z : p_\varphi(x, y) \Rightarrow \varphi(x, y, z)) \wedge (\forall x : \forall y : \forall z : p_\varphi(x, y) \Leftarrow \varphi(x, y, z)). \end{aligned}$$

Note that both formulas in this conjunction can again be grounded using the techniques we discussed earlier, as they contain (maximally) an alternation of two different quantifier types. p_φ is simply treated as a new partially interpreted predicate.

Similarly, consider

$$\psi_2 \Leftrightarrow \forall x : \exists y : \forall z : \varphi(x, y, z).$$

This time, we introduce p_φ as follows:

$$p_\varphi(x, y) \Leftrightarrow \forall z : \varphi(x, y, z).$$

Now the original ψ_2 is equivalent to $\forall x : \exists y : p_\varphi(x, y)$, which again can be grounded using the techniques described earlier. We add the following to the theory:

$$\begin{aligned} & \forall x : \forall y : (p_\varphi(x, y) \Leftrightarrow \forall z : \varphi(x, y, z)) \\ \Leftrightarrow & \forall x : \forall y : (p_\varphi(x, y) \Rightarrow \forall z : \varphi(x, y, z)) \quad \wedge \quad \forall x : \forall y : (p_\varphi(x, y) \Leftarrow \forall z : \varphi(x, y, z)) \\ \Leftrightarrow & (\forall x : \forall y : \forall z : p_\varphi(x, y) \Rightarrow \varphi(x, y, z)) \quad \wedge \quad (\forall x : \forall y : \exists z : p_\varphi(x, y) \Leftarrow \varphi(x, y, z)), \end{aligned}$$

both of which again can be grounded as described earlier.

In general, for formulas with even more than 3 (alternating) quantifiers, the following steps can be followed, effectively applying the procedures described for ψ_1 and ψ_2 above recursively. Let the formula ψ to be grounded be $Q_1x^1 : Q_2x^2 : \dots Q_nx^n : \varphi(x^1, x^2, \dots, x^n)$, with $Q_i \in \{\exists, \forall\}$ and quantifier types always alternating.

1. Take the deepest quantifier Q_n and the quantified formula φ , and define a new predicate $p_\varphi \Leftrightarrow Q_nx^n : \varphi$, which is free in all variables apart from x^n .
2. Substitute this new predicate in the original ψ , transforming it to an equivalent

$$\psi' \Leftrightarrow Q_1x^1 : Q_2x^2 : \dots Q_{n-1}x^{n-1} : p_\varphi(x^1, \dots, x^{n-1}).$$

3. Add the following formula to the theory:

$$\forall x^1 : \dots \forall x^{n-1} : (p_\varphi(x^1, \dots, x^{n-1}) \Leftrightarrow Q_nx^n : \varphi(x^1, \dots, x^n)),$$

which, depending on the type of Q_n , by a reasoning as described earlier in this section, reduces to:

- For $Q_n = \exists$:

$$\begin{aligned} & \forall x^1 : \dots \forall x^{n-1} : \exists x^n : (p_\varphi(x^1, \dots, x^{n-1}) \Rightarrow \varphi(x^1, \dots, x^n)) \\ \wedge & \quad \forall x^1 : \dots \forall x^n : (p_\varphi(x^1, \dots, x^{n-1}) \Leftarrow \varphi(x^1, \dots, x^n)). \end{aligned}$$

- For $Q_n = \forall$:

$$\begin{aligned} & \forall x^1 : \dots \forall x^n : (p_\varphi(x^1, \dots, x^{n-1}) \Rightarrow \varphi(x^1, \dots, x^n)) \\ \wedge & \quad \forall x^1 : \dots \forall x^{n-1} : \exists x^n : (p_\varphi(x^1, \dots, x^{n-1}) \Leftarrow \varphi(x^1, \dots, x^n)). \end{aligned}$$

4. In case ψ' contains more than 2 alternating quantifiers, repeat the procedure above with ψ' in the role of ψ .

Given that implementing this reasoning is more of an exercise in pattern matching on the structure of the formulas in question, than contributing to the aim of this thesis, as stipulated in 2.7, the implementation of the general case of more than 2 mixed quantifiers was deemed outside of the scope of this project.

3.3.7 Formulas not in Prenex Normal Form

We started Section 3.3.2 by stating the assumption that all quantifiers had been made explicit in the formulas mentioned and quantifiers were in front, which means that all paragraphs on quantified formulas so far, have been dealing with *prenex normal form* (PNF). While formulas can always be converted to PNF, doing so would introduce a lot of redundancy.

As an example, consider $\forall x : (\varphi(x) \vee \forall y : \psi(x, y))$, and say that $x \in \{x_1, x_2, x_3\}$ and $y \in \{y_1, y_2, y_3\}$. The formula grounds to:

$$\begin{aligned} & \varphi(x_1) \vee (\psi(x_1, y_1) \wedge (\psi(x_1, y_2) \wedge (\psi(x_1, y_3))) \\ & \wedge \varphi(x_2) \vee (\psi(x_2, y_1) \wedge (\psi(x_2, y_2) \wedge (\psi(x_2, y_3))) \\ & \wedge \varphi(x_3) \vee (\psi(x_3, y_1) \wedge (\psi(x_3, y_2) \wedge (\psi(x_3, y_3))). \end{aligned}$$

The equivalent PNF version of the formula is $\forall x : \forall y : (\varphi(x) \vee \psi(x, y))$. This formula grounds to the following:

$$\begin{aligned} & (\varphi(x_1) \vee \psi(x_1, y_1)) \\ & \wedge (\varphi(x_1) \vee \psi(x_1, y_2)) \\ & \wedge (\varphi(x_1) \vee \psi(x_1, y_3)) \\ & \wedge \dots \\ & \wedge (\varphi(x_3) \vee \psi(x_3, y_3)), \end{aligned}$$

a conjunction of in total 9 subformulas. Note that e.g. $\varphi(x_1)$ is repeated thrice, unlike in the grounding from the original formula not in PNF, which did avoid this redundancy. Using the distributive law, it can be seen they are indeed equivalent.

To avoid this blow-up should we transform all formulas to PNF, we propose the following to ground $\forall x : (\varphi(x) \vee \forall y : \psi(x, y))$. Similar to the approach of 3.3.6, we introduce a new predicate $p(x) \Leftrightarrow \forall y : \psi(x, y)$. The equivalent $\forall x : (\varphi(x) \vee p(x))$ can be grounded as before. We also add to the theory $\forall x : (p(x) \Leftrightarrow \forall y : \psi(x, y))$, which is, by the reasoning as explained in 3.3.6, equivalent to

$$(\forall x : \forall y : p(x) \Rightarrow \psi(x, y)) \quad \wedge \quad (\forall x : \exists y : p(x) \Leftarrow \psi(x, y)),$$

both of which, again, only contain (maximally) two alternating quantifiers, and can therefore be grounded as before.

This approach can be generalised to all formulas *not* in PNF. With a procedure very similar to the one explained in 3.3.6, one can introduce new predicates, starting from the most deeply nested subformulas. This until the main formula *is* in PNF. Again, as this involves more formula preprocessing based on the structure of the formulas in the theory, the high-level theoretical description presented in this section was deemed sufficient within the scope of this thesis, and implementation has been omitted.

3.4 The Solver

Once the full grounding has been computed as a set of constraints in the SMTLIB format, it is handed to the solver, which checks whether the theory is SAT and performs model expansion. As described in Section 2.3, we use the Z3 solver *as is*.

A new, a vast, and a powerful language is developed for the future use of analysis, in which to wield its truths so that these may become of more speedy and accurate practical application for the purposes of mankind than the means hitherto in our possession have rendered possible.

Ada Lovelace

4

Implementation

In this short chapter, we will discuss some more technical details and design choices of the implementation. The reader who wishes to concentrate on the conceptual, rather than implementation side of the system, can skip this chapter without loss of continuity.¹ Note that the first four sections of this chapter follow again the same phases the system executes: (1) parsing the file, (2) building the database, (3) grounding the formulas in the theory, (4) presenting the grounding to the solver. The final section explains the structure of the Git repository where the code of the implementation can be found, to be consulted at <https://github.com/nvinckie/Master-Thesis>.

4.1 Parsing

As a starting point, a Python-based project² shared by Simon Vandeveld was used, which makes use of Treesitter [Brunsfeld et al.], a Python parsing library. Application of this yields a syntax tree which is traversed to process the IDP file. An instance of the class `Query_KB` is created which keeps track of all the elements: types, functions, predicates, formulas in the theory etc.³ All types of formulas are subclasses of the `FOFormula` parent class.

¹Which was the rationale for having this chapter detailing the implementation, as not to clutter Chapter 3, which focuses on the conceptual side.

²A personally developed variant of the IDP-Z3 solver.

³Most of the code to achieve this was part of the starting project and not our own work.

4.2 Building the Database

A connection to either the on-disk SQLite database or in-memory DuckDB database is maintained via an instance of the `DB_Manager` class in the `database_manager.py` module.

As described in 3.2.1, queries are generated to:

- Create tables for types;
- Insert values into these tables;
- Create tables for functions and predicates;
- Insert values into tables for fully interpreted predicates (in which case this predicate is flagged as fully interpreted).

These queries are then applied to the database.

We indicate a small technicality regarding the table names. Because of the case-insensitiveness of both SQLite and DuckDB table names, types, predicates and functions with the same name but different casing, have to be avoided. This haphazard technicality is easily avoided (and did not occur in any of the cases of DIRT, refer to Chapter 5) in all but the most unfortunate of IDP files. One of which is exactly the toy example we’ve been using throughout, which distinguishes between predicates P and p ...⁴

Once the database has been initialised, the quantified and unquantified fact queries are generated and applied. For the quantified kind, we exploit the `ground` function of a quantified formula.⁵ Note that we also flag whether fact queries have been generated at all, because if not, while grounding, the database does not have to be queried for facts about partially interpreted predicates and functions. Formulas that have been used to generate a fact query, are flagged and will be ignored while grounding, as the information they provide has been stored in the database.

4.3 Grounding the Theory

To initiate the grounding phase, the main grounding function `to_ground_smt_lib` is called and the resulting grounding written to file.⁶ This comprises three steps, in this order:

1. Declaring function and predicate types;
2. Grounding the formulas in the theory that were not used to generate fact queries;
3. Writing the information from the tables of functions and partially interpreted predicates back to the grounding.

⁴During implementing and testing, these had to be named differently in the IDP file.

⁵Confusingly enough, as we are not grounding the theory yet.

⁶Easier for debugging and passing the grounding on to the solver, refer to the next section.

The reason for this order is that if the grounding of one of the formulas in step 2 turns out to be **false**, the procedure is halted and the entire grounding is merely (**assert false**), so the solver (refer to the next section) has no work to do and the remainder of step 2, and step 3 can be skipped altogether.

Grounding **unquantified** formulas (refer to Section 3.3.1) is relatively straightforward. E.g. formula of the form $\varphi(a_1) \wedge (f(a_1) = b_1)$, an instance from the **And**-class, will give rise to a constraint (**and** <g1> <g2>) with the two groundings established at the deeper levels, with simplification in case either is **true** or **false**. At the deepest level, we encounter instances of the class **AppliedSymbol** (predicate and function applications), which are grounded as follows:

- Fully interpreted predicates are simply queried for and yield **true** or **false**;
- Partially interpreted predicates and function (in)equalities are queried for and in case a truth value is found in the database (the result of a fact query), these yield **true** or **false**. If not, the expression is translated to SMTLIB syntax (refer to Section 2.5). Note that, as stated in the previous section, we only query for facts if it has been flagged that fact queries have been generated (if not, this would be a waste of time).

Quantified formulas are more complicated to ground. This is done in the class **Quantification**. First, the deepest quantified subformula has to be determined, e.g. $\varphi(x, y)$ in a formula $\forall x : \exists y : \varphi(x, y)$. Note that this hinges on the PNF of the formula.

Grounding a quantified formula naively is implemented in this class and is relatively straightforward. The formula is grounded over the full Cartesian product of the types involved.

Grounding quantified formulas for different quantifier successions, in the ways explained in Sections 3.3.2 to 3.3.5, is of course quite complex. To keep track of the information about the quantification, we make use of the class **QuantifiedVars**. Instances of this class hold, for all free variables in a quantified formula, dictionaries to link them to their quantifier type ($\exists$ or $\forall$) and their type.⁷

Below, we will explain in a bit more detail how grounding formulas of the type $\forall \vec{x} : \varphi(\vec{x})$ (so only universal quantifiers) and $\forall x : \exists y : \varphi(x, y)$ have been implemented. The other two types are analogous.

In case a formula is quantified with only universal quantifiers, first, the fully interpreted predicates $P_1, \dots, P_n$ mentioned in it are established. Then, by making use of the representations of j (from 0 to $2^n - 1$) as binary strings, the set P' (refer to Section 3.3.2 for terminology) is computed. Formulas φ'_j are generated for each j , and simplified. Note that φ'_j that simplify to $\perp$ are considered first to potentially spot a trivially UNSAT situation early. To have tighter control, the formulas φ'_j are constructed as instances of the class **ExpressionTree**, with the formula's logical connectives and equality as nodes and atoms as leaves. A formula of the form $P'_j \Rightarrow \varphi'_j$ is then grounded by querying for the SAT-set S_j of P'_j , and grounding φ'_j over S_j . For each tuple of constants in S_j , the quantified variables are *materialised* and the **ExpressionTree** evaluated for this particular tuple of constants. Similarly to grounding non-quantified formulas, e.g. an “and” tree node will be grounded to (**and** <g1> <g2>) with g1 and g2 the groundings of its two

⁷Functionality which needs this information to ground a quantified formula has been implemented in this class as well: generating the query to find a SAT-set, making an **ExpressionTree** (see below), generating the grounding for an **ExpressionTree** etc.

children, and simplification is performed on the fly. At the deepest level, **AppliedSymbols** are encountered again, which are grounded similarly, but now for the materialised constants.

Grounding a formula of the form $\forall x : \exists y : \varphi(x, y)$ is similar to the above, but extra “admin” has to be performed, in the form of keeping track of additional lists and dictionaries (refer to Section 3.3.4). In the following, indices j are always saved in their bitstring representation.

- **J_NT**: a list representing J_{NT} ;
- **P_prime**: a dictionary mapping j to P'_j ;
- **S_j_s**: a dictionary mapping j to the SAT-set S_j of φ'_j , which is needed to find sets S_j^x ;
- **S_top**: a list representing $S_{\top}$;
- **to_ground_triv**: a list with those j for which $\varphi'_j \Leftrightarrow \top$;
- **to_ground_real**: a list with those j for which $\varphi'_j \not\Leftrightarrow \top$;
- **X_NT**: a list representing X_{NT} .

First, trivial cases are established and **X_NT** computed. Should this list be empty, SAT is returned immediately (the first edge-case in Section 3.3.4). If not, the grounding is calculated, potentially halting early in case the second edge-case is encountered. Note that it does involve some manual iteration of lists, making this and the other mixed quantifier succession more complicated than the kinds only involving one quantifier type (as will be discussed in Section 5.2).

4.4 The Solver

The grounding is read from file and handed to the solver. The only optimisation is that when the grounding contains either:

- a statement (**assert false**), i.e. the theory is trivially UNSAT,
- or *only* statements (**assert true**), i.e. the theory is trivially SAT,

the solver is not defined at all and respectively “Trivially (UN)SAT” is returned.

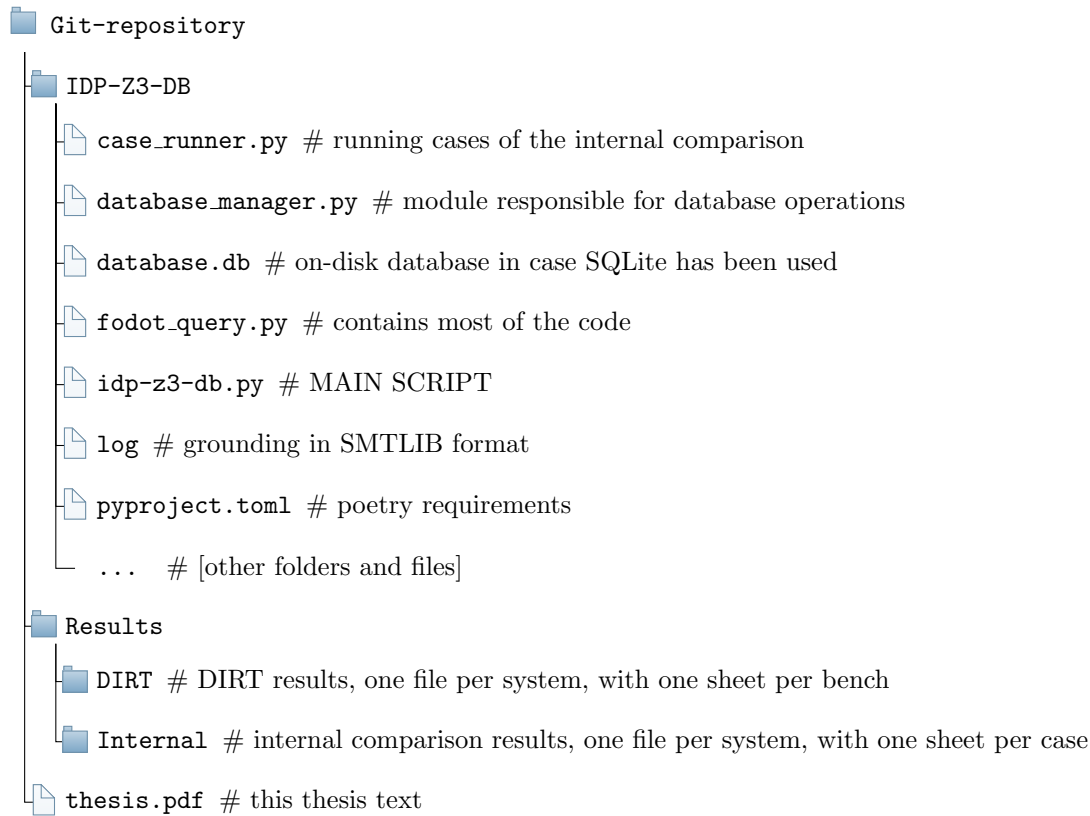
Barring trivial situations, as mentioned before, we use the **Z3** solver⁸ “as is”. It takes the grounding in SMTLIB format and computes one model.

One a technical note, we remark that while the definition and enumeration of the types can be parsed from the SMTLIB format as well, this proved to be very slow, and instead they were declared manually to the solver.

⁸More exactly, its Python module, refer to `z3py` Namespace Reference.

4.5 Structure of the Git Repository

Below, we indicate the structure of the Git repository.



The requirements for the code have, just as mentioned in the Git repository of IDP-Z3 [Vandevelde], been managed using the package manager Poetry.

Running the main script from the command line is done by using

```
<poetry-virtual-environment-binary> idp-z3-db.py <idp-file> <db-manager>
```

The user will have to locate the virtual Poetry environment binary, in our case it can be found at `/home/.cache/pypoetry/virtualenvs/duckdbdep-9GR9n2aE-py3.12/bin/python3.12`. The `<db-manager>` argument can be either `sqlite`, `duckdb` or `naive`.

To run the cases described in Section 5.2, the user still has to update their environment (refer to the code, the `run_case` function and function call in the final line of the `case_runner.py` file).

This can be a bit fiddly, in the same sense that key-hole surgery and hostage negotiations can be a bit fiddly.

James May

5

Testing and Comparing

To test the efficiency of the IDP-Z3-DB system explained in the previous two chapters, we propose an “external” and an “internal” comparison.

In the external comparison, the system will be tested and compared against other grounders and solvers, namely IDP-Z3 and SLI. This will be done using the DIRT framework and benchmarks (Section 5.1).

In the internal comparison (Section 5.2), different kinds of theories will be tested within the system itself, and we will attempt to find (and explain why) on which kinds of cases the system performs better or worse.

For each, we have to consider the difference between using DuckDB or SQLite as the RDBMS. We will refer to them as systems (D) and (L) respectively.¹

5.1 External Comparison

5.1.1 DIRT

To compare our grounding system against others, we will use the DIRT benchmark framework by Van Laer et al. [2025], designed to compare grounders. This has the advantage that timings of the different benchmarks can be compared consistently. There are two systems we wanted to compare against:

¹We use these to refer to the systems as a whole, but still use “DuckDB” and “SQLite” to refer to specifically their respective database managers.

- *SLI*, a grounder by Van Laer et al. [2024] for IDP, using bitvectors to compute SAT-sets while grounding, which is an approach not dissimilar to ours and therefore an interesting one to compare against. It has been implemented in Rust, a compiled language, so we expect it to outperform our system when it comes to processing long lists of tuples to find a grounding.
- *IDP-Z3*, the system working at the back-end of the online [IDP-Z3] interface. Van Laer et al. [2025] concluded that SLI outperforms this system by several orders of magnitude, and our system showed similar results, permitting no meaningful analysis², so we will exclude its results in Section 5.1.2.

We will test all the benches that make use of only concepts from FO[Core], namely the following scenarios.

- *CommonItem*: Two fully interpreted predicates, P and Q , and a formula checking whether there is an overlap: $\exists x : P(x) \wedge Q(x)$. This scenario consists of two benches: both a SAT and UNSAT version.
- *CompleteSets*: A complement to the former, to check whether the two predicates together cover a full domain: $\forall x : P(x) \vee Q(x)$. There is again a SAT and UNSAT version.
- *GraphColouring*: Similar to the map-colouring problem, this comprises a fully interpreted predicate E for connectivity and a `colourOf` function from vertices to colours, with as a constraint $\forall x, y : E(x, y) \Rightarrow (\text{colourOf}(x) \neq \text{colourOf}(y))$.
- *TriangleGraph*: Searching for triangles in a graph. This scenario has multiple different benches, as described in [Van Laer et al., 2025]. Some of them make use of an inductive definition for the triangle predicate, but in a way such that it can be eliminated as an equivalence (refer to Section 3.1).
 - *TriangleGraphCheckSat* and *TriangleGraphCheckUnsat*: Both the ternary predicate P indicating a triangle between vertices, and the edge predicate E , are fully interpreted and the aim is to check whether P correctly captures exactly the triangles in the graph (yes and no, respectively).
 - *TriangleGraphConstructAtleast*: The edge predicate E is fully interpreted, the ternary triangle predicate p is not and should cover at least all triangles in the graph.
 - *TriangleGraphConstructGraph*: Conversely, now the triangle predicate is fully interpreted, and the edge predicate needs to be constructed in such a way that the graph contains exactly the triangles described.
 - *TriangleGraphContainsAll*: A one-sided variant of the “check UNSAT” benchmark, so again both the edge predicate E and triangle predicate P are fully interpreted, and it is checked whether P covers all the triangles defined by E .
 - *TriangleGraphFindAll*: A variant of the “at least” benchmark, but now p should cover *exactly* all triangles.
 - *TriangleGraphSubset*: Here, a predicate S of selected vertices of a graph is given, together with the edge predicate E . The aim is to find a triangle predicate p that covers at least all the vertices selected by S .

²Other than “it’s really slow” ...

5.1.2 Results and Interpretation

All tests have been run on an Ubuntu 24.04 system equipped with an Intel Core Ultra 7 155U CPU and with a limit of 16 GB of RAM available. The benches were run with a 600 second timeout setting. Below we summarise and analyse the results. Full timings have been included in the Git repository. Should the reader not wish to read these details, they can, without loss of continuity, progress to the conclusions of running the DIRT benches, presented in Section 5.1.3.

CommonItem and CompleteSets

The results for these benches are summarised in the table below, each time the mean total run time in seconds $\pm$ one standard deviation. Each bench consisted of 50 instances.

Bench	SLI	(D)	(L)
<i>CommonItemSAT</i>	4.5 ± 2.0	4.2 ± 1.8	1.8 ± 0.7
<i>CommonItemUNSAT</i>	2.9 ± 1.7	2.9 ± 1.6	1.8 ± 1.0
<i>CompleteSetsSAT</i>	4.0 ± 1.8	4.3 ± 1.8	1.8 ± 0.8
<i>CompleteSetsUNSAT</i>	3.5 ± 1.9	3.8 ± 2.0	1.6 ± 0.8

These results reveal that for these benches, the system with (D) is similar in performance to SLI. The (L) version outperforms SLI though, with a factor of roughly 2.

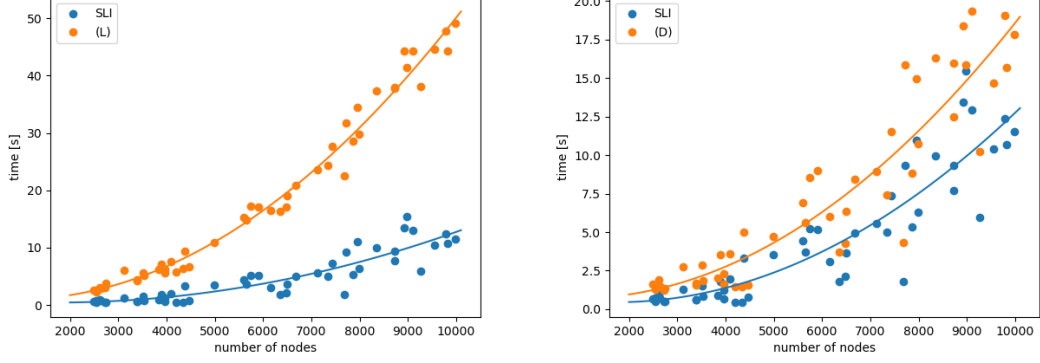
We note that manual inspection³ reveals that almost all of the time is spent setting up the database, not in actually grounding the formula. This is easy to explain: these benches have *only* fully interpreted predicates and the grounding of the formula is therefore trivial: either **true** or **false**. E.g. for *CompleteSets*,⁴ once the database has been set up, the system just has to check whether the SAT-set of $\neg P(x) \wedge \neg Q(x)$ is or *isn't* empty, to establish SAT or UNSAT, respectively. The variability is solely explained by the number of constants and the sparseness of P and Q , which affect the time it takes to set up the database.

GraphColouring

For this bench, unlike the previous four, actual grounding and solving work has to be done. In [Van Laer et al., 2025], it is mentioned that 0.1% to 1% of all potential edges are present in the graph, so the amount of grounding work is still quadratic w.r.t. the number of vertices. The graphs below show the results of comparing (L) and (D) against SLI, together with the best-fitting quadratic polynomial.

³DIRT does not make a distinction between *parsing* and *grounding* phases, but takes both together as *presolve* time.

⁴The situation for *CommonItem* is symmetric.



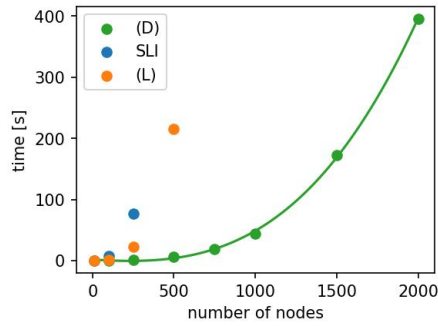
Note that the spread of the solving times around the expected quadratic behaviour, can be explained by the different densities of the graphs.

We notice system (L) is outperformed by SLI by a factor of roughly 5, while system (D) is outperformed by a factor between 1.5 and 2. The differences can be explained because in this bench we have to process the result set of a query, which DuckDB can potentially do more efficiently than SQLite, a fact to which we will get back in Section 5.2.2.

TriangleGraph: SAT – UNSAT – ContainsAll

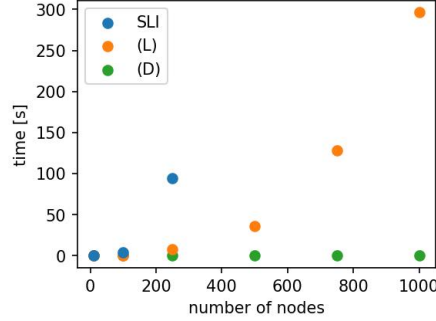
The reason why we discuss these three benches together, is because just like the first 4, all predicates are fully interpreted.

For **TriangleGraphSAT**, the results are shown below. The best-fitting third degree polynomial has been plotted for one case, to show the cubic behaviour we expect to see, as Van Laer et al. [2024] mention that the edge predicate is filled with $n/3$ tuples, with n the number of nodes. No dot means the system encountered a timeout.



While SLI times out on instances with 500 nodes, (L) only does so on 750 and (D) – helped by query optimisation – only on 4000.

For **TriangleGraphUNSAT**, the results are similar for SLI and (L) in comparison to the SAT case.

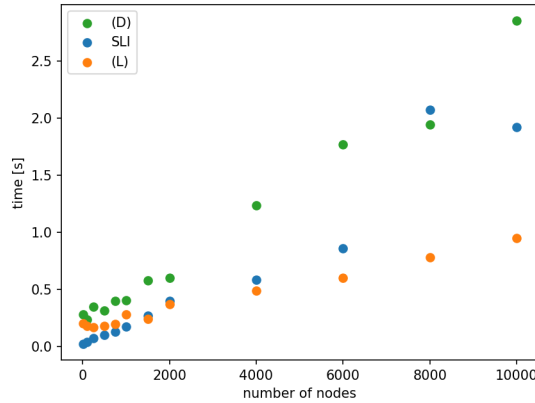


System (D), however, successfully solves all instances in no time, the one with 100,000 vertices clocking in at 6.6 seconds! In order to explain this, we have to deeper examine this bench. With P as the triangle predicate and E the edge-sharing predicate, the two formulas to verify are

$$\begin{aligned} \forall x, y, z : & (E(x, y) \wedge E(y, z) \wedge E(x, z)) \Rightarrow P(x, y, z), \\ \forall x, y, z : & P(x, y, z) \Rightarrow (E(x, y) \wedge E(y, z) \wedge E(x, z)). \end{aligned}$$

The first is the “easy direction”: only $E(x, y) \wedge E(y, z) \wedge E(x, z) \wedge \neg P(x, y, z) \Rightarrow \perp$ needs to be checked (if there is a tuple satisfying the antecedent, the theory is UNSAT), and all systems do this relatively quickly. For the converse, however, it is intuitively clear that this is harder: any of the edged of a triangle could fail to be present. But for this particular bench, P is the empty predicate (i.e. known to not hold on all tuples), so the second condition is trivially satisfied. The query optimiser of DuckDB clearly spots this, yielding an empty result set, and quickly reveals that there is no tuple violating the second formula (and the unsatisfiability of the theory is caused by the first formula).

As for **ContainsAll**, this is exactly only the “easy direction” explained above. As the instances are indeed UNSAT, it is no surprise then that all systems perform well on spotting this early and avoiding the $O(n^3)$ blow-up. For the sake of clarity, the 50,000 and 100,000 vertex benches have been omitted in the plot below. For the 100,000 one, SLI, (L) and (D) finished in respectively 20.0, 8.0 and 26.7 seconds.



TriangleGraph: ConstructAtleast – ConstructGraph – FindAll – GraphSubset

ConstructAtleast is processed smoothly by SLI, with 10.8 seconds on the largest instance of 100,000 vertices. In our system, its formula

$$\forall x, y, z : (E(x, y) \wedge E(y, z) \wedge E(x, z)) \Rightarrow p(x, y, z),$$

with finding the triangle predicate p as a goal, is the subject of a fact query. Processing these and updating the database, however, takes time.⁵ (L) timed out on 50,000 vertices, solving the instance on 10,000 in 133 seconds. (D) was slightly more efficient, but also runs out of steam on the largest instance, solving the second-largest, on 50,000 vertices, in 254 seconds.

For **ConstructGraph**, **FindAll** and **Subset** we expect a tough challenge, because the combinatorial explosion cannot be avoided. Take e.g. the formulas to construct a graph:

$$\begin{aligned} \forall x, y, z : P(x, y, z) &\Rightarrow (e(x, y) \wedge e(y, z) \wedge e(x, z)), \\ \forall x, y, z : (e(x, y) \wedge e(y, z) \wedge e(x, z)) &\Rightarrow P(x, y, z). \end{aligned}$$

While the first gives rise to a fact query, filling up the edge predicate with what it *at least* has to contain, the second is equivalent to

$$\forall x, y, z : \neg P(x, y, z) \Rightarrow (\neg e(x, y) \vee \neg e(y, z) \vee \neg e(x, z)),$$

yielding a conjunction (over the SAT-set of $\neg P$) of disjunctions (of the three negations in the consequent).

In all three, SLI failed at the 250-vertex instance, not because of a timeout, but because the system ran out of memory. All 100-vertex instances were solved in between 14 and 24 seconds.

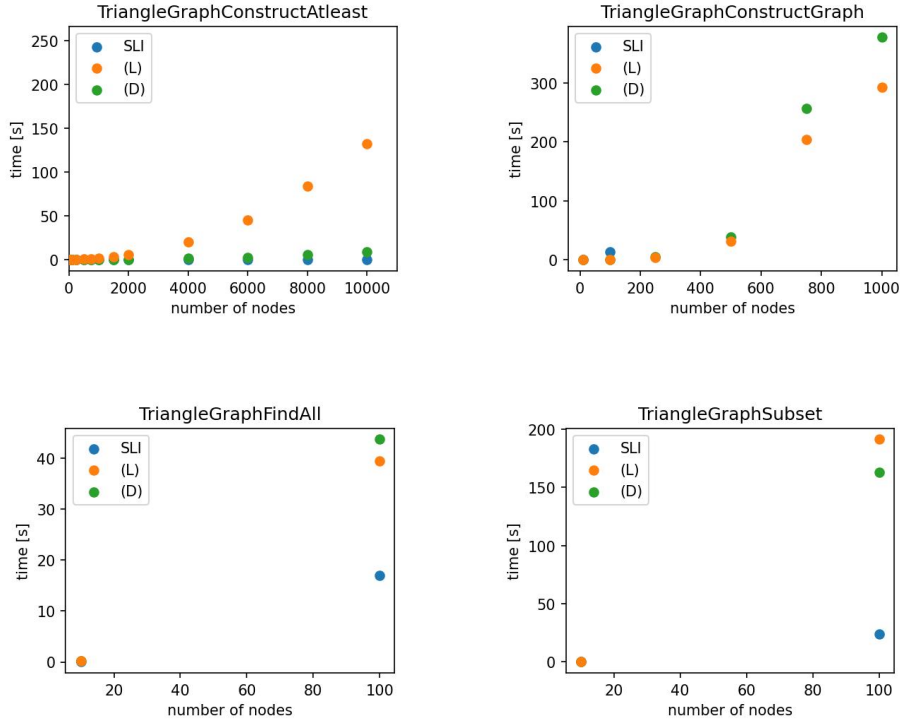
System (L) performed better on the **ConstructGraph** bench, only timing out at the 1500 instance, solving the 1000 one in 292 seconds. On **FindAll** and **Subset**, however, it also timed out on the 250 instance, solving the 100 ones in respectively 39 and 191 seconds.

System (D) had a similar performance on **ConstructGraph**, solving the 1000-vertex instance in 377 seconds and timing out on the next one. Also **FindAll** and **Subset** are similar, solving the 100 instance in 44 and 163 seconds respectively, before timing out on the 250 one.

We summarise the timings for these four benches in the graphs below.⁶ As before, no dot means the respective system encountered a timeout.

⁵For the sake of the experiment, we tried not generating fact queries and grounding the formula just like that, which yielded even slower results.

⁶In the TriangleGraphConstructAtLeast bench, the x -axis has been limited to 10 000 vertices, for a more meaningful comparison.



5.1.3 Overall Conclusion of the DIRT Benches

Having discussed all the benches, we can draw a couple of conclusions.

In case all of the predicates are fully interpreted, and it has to be established whether the theory is (UN)SAT, our system is comparable to or better than SLI. System (D) is preferred when a more-than-linear blowup is to be expected, or in an edge-case (like in TriangleGraphUNSAT) where DuckDB's query optimiser comes to the rescue. It has more overhead than SQLite, however, to define types and predicates, which is why in some benches (e.g. the first four), system (L) performs better.

When "actual" grounding work has to be done, results are mixed. Especially when large iterations have to be done over result sets (passing the boundary between the database manager and Python), or large fact queries processed, our system is similar (GraphColouring) to or slower (ConstructAtLeast) than SLI.

But sometimes generating fact queries to update the database for partially interpreted predicates, makes our system faster than not doing so, and in at least one bench (TriangleGraphConstructGraph) made it way quicker than SLI.

Overall, the two versions (D) and (L) of IDP-Z3-DB are competitive, but for which instances they are better and for which worse, depends on the structure and the kind of formulas in the theory.

5.2 Internal Comparison

While the external comparison already yielded some insight into the efficiency of the system, in this section, the aim is to test on which kinds of theories and structures the system performs well, and which others it struggles with, and try and give an explanation to differences observed.

5.2.1 Setting Up the Test Cases

In order to establish a useful suite of test formulas, different dimensions have to be considered, which we will denote (A), (B), (C) and (D).

(A) The formula in the theory:

- Types of connectives and quantifiers they make use of;
- Number of fully vs. partially interpreted predicates and functions involved.

For this dimension, we will consider two formulas per quantifier succession, numbered as follows:

Cases	Quantifiers
(A1)-(A2)	$\forall x : \forall y :$
(A3)-(A4)	$\exists x : \exists y :$
(A5)-(A6)	$\forall x : \exists y :$
(A7)-(A8)	$\exists x : \forall y :$

(B) Sparseness of the information:

- For fully interpreted predicates: sparseness of the structure;
- For partially interpreted predicates and functions: the number of truth values established via fact queries (refer to Section 3.2.4).

When discussing the formulas in more detail, for this (B) dimension, we will use codes **t** for trivial, **s** for sparse and **f** for dense.⁷

(C) The size(s) of the domain(s) involved. This dimension will be discussed when the test results are examined.

(D) SAT vs. UNSAT. This dimension is integrated within the description of the test cases. The external comparison showed that the system is good at picking out the trivially (UN)SAT theories based solely on the *fully interpreted predicates*, consulted from the database. Therefore, we concentrate on cases that are trivially (UN)SAT because of additional information we have about

⁷In an earlier version we used the term “full”, before realising *dense* is a better term... Although in some cases, the predicate(s) will be so dense they indeed cover the full domain.

the *partially interpreted predicates* (i.e. processed by fact queries). For the mixed quantifiers, however, as these were not tested in DIRT, we will include some trivially (UN)SAT cases based on the fully interpreted predicates as well.

For the sake of clarity, the actual formulas and their density will be discussed together with their results and interpretation, and we provide some justification. In the tables below it may seem only one formula is in play, but in case a partially interpreted predicate, say $q(x, y)$, needs to be “filled up”, we will do this with a formula

$$\forall x, y : S(x, y) \Rightarrow q(x, y),$$

which will give rise to a fact query, filling up q with the tuples which have been selected by a predicate S (which is defined in the structure).

In the next section we introduce the test cases and discuss the results in detail. Should the reader wish to do so, this detailed analysis can be skipped without loss of continuity, as we provide a conclusion in Section 5.2.3.

5.2.2 Results and Interpretation

Below we discuss the results of the internal comparison, again run with a 600 second timeout setting. All combinations of test cases have been tested against the naive grounding method, which has also been implemented (refer to the introduction of Section 3.3). We refer to it as (N). Note that it does make use of the database⁸ to write the information of the fully interpreted predicates to the file containing the SMT grounding, both positively (e.g. P) and negatively (e.g. $(A \times B) \setminus P$).

We refer to the Git repository for the folder with the full results, and explain the most relevant observations in this section. When we speak of “an instance on n constants”, this means both A and B have n constants and the size of the solution space is n^2 . To generate increasingly larger instances, we used $n = 10i^2$, for $i = 1, \dots, 15$. In tables, noteworthy values are indicated in bold, these are also timings mentioned in the analysis.⁹

Important to note is that what is referred to as *parsing* time, is broader than text parsing: it is the phase in which the IDP file is parsed (in the narrow sense), *plus* the time for setting up the database¹⁰: defining the tables, filling them with tuples and processing fact queries, i.e. all the pre-grounding work.

⁸The SQLite version.

⁹Tables with timings are only included when comparison is enlightening, if not, results and timings are simply discussed in the text.

¹⁰Refer to Sections 3.2.1, 3.2.2 and 3.2.4.

(A1)	$\forall x \in A : \forall y \in B : P(x, y) \vee q(x, y)$
Should avoid a lot of grounding work in case P is dense.	
(Bs)	Sparse P ($\approx 5\%$ of $A \times B$) and q fully unknown.
(Bf)	Dense P ($\approx 99.9\%$ of $A \times B$) and q fully unknown.
(Bt)	Sparse P ($\approx 5\%$ of $A \times B$) and q well-known through fact queries ($\approx 95\%$ of $A \times B$). $P \cup q = A \times B$, so the theory is trivially SAT.

For the **(Bs)** variant, with a sparse P , the run times on the 810-constant instance (on which all three finished), have been listed below.

db-system	parsing	grounding	solving	total
Naive	0.3052	510.6113	14.1012	525.0177
DuckDB	0.5527	1.8706	4.5719	6.9952
SQLite	0.3285	569.0115	4.6188	573.9588

We notice that system (D) is quite fast, with roughly 30% of the time taken up by parsing and grounding, the remaining time solving, which makes sense as even the minimal grounding is large, still leaving a lot of work to the solver. (L) takes 50 to 100 times more time, because SQLite is less optimised to fetch the result set of the query involved, to such an extent that it is equally slow as (N)! Bluntly grounding the entire formula over the full $A \times B$ is equally efficient as querying with SQLite for where $\neg P(x, y)$ holds, and grounding $q(x, y)$ by iterating over that result set. This is because in this instance, the formula to ground is equivalent to $\forall x \forall y \neg P(x, y) \Rightarrow q(x, y)$, the antecedent of which is translated to a query

```
SELECT A.A, B.B FROM A,B WHERE
  (NOT (A.A,B.B) IN (SELECT P.A,P.B FROM P))
```

SQLite will generate the Cartesian product $A \times B$ and only then compare that to the result of the subquery, while DuckDB is much quicker to fetch the result of this query.

For a dense P , however, in **(Bf)**, both system (D) and (L) outperform (N), but close inspection of the times on e.g. the instance on 1000 constants, reveals important differences:

db-system	parsing	grounding	solving	total
Naive	7.9912	40.3701	13.7131	62.0744
DuckDB	15.0733	0.5116	0.0686	15.6535
SQLite	6.4156	32.3497	0.064	38.8293

It is clear that SQLite still loses a lot of time querying for the result set, despite it containing only 1000 elements¹¹, illustrating the observation that even if the final result is small, SQLite is less optimised if the intermediate result can be quite large. DuckDB, however, loses all its time

¹¹ $\neg P(x, y)$ holds on 0.1% of the 1000² tuples in $A \times B$

in setting up the database, actual grounding is incredibly quick. This confirms an observation we hinted at in Section 5.1.2: DuckDB is slower at setting up the database, but better optimised for querying it and processing the results. Note how both avoid the large solving time by keeping the grounding as small as possible.

For **(Bt)**, it is system (L) that is faster than (N), by a factor 2 to 3. For system (D), the results are disastrous, even timing out before (N) does. Below, we list the timings for the 490-constant case, the last one on which they all finish.

db-system	parsing	grounding	solving	total
Naive	1.7524	158.5922	9.4389	169.7835
DuckDB	7.7994	327.1545	1.3288	336.2827
SQLite	2.3329	71.0275	1.529	74.8894

While DuckDB is more efficient than SQLite at processing complexer queries, querying every time for a single potential fact stored in the database causes more overhead for (D) than for (L). We hypothesise that this is because there is a fixed amount of overhead time to execute any query, which is less for the lightweight SQLite than for DuckDB. Switching off fact query generation and running (D), however, yields fast run times, roughly three times those of its (Bs) results.¹² This confirms a suspicion raised in Section 5.1.2: fact queries can help, but mostly when they allow the system to spot a triviality (and therefore halt) early, if not, a lot of time is still spent querying for these facts.

(A2)	$\forall x \in A : \forall y \in B : P(x, y) \vee Q(x, y) \vee f(x, y) = b_0$
	Should avoid a lot of grounding work in case $P \cup Q$ covers a lot of $A \times B$.
(Bt)	Sparse P and Q (covering the same constants and $\approx 5\%$ of $A \times B$), but $f(x, y) \neq b_0$ holding for the remaining part, making the theory trivially UNSAT.
(Bs)	Sparse P and Q (covering the same constants and $\approx 5\%$ of $A \times B$).
(Bf)	Dense P and Q (disjoint and covering $\approx 49.9\%$ of $A \times B$ each).

We see the final observation of the last paragraph confirmed by **(Bt)**. The naive version cannot spot the inconsistency, timing out on the 810 instance, while our systems spot it immediately. Again, (D) loses more time setting up the database than (L), confirmed by the run times on e.g. 1000 constants:

db-system	parsing	grounding	solving	total
DuckDB	23.3801	0.4643	0.0061	23.8505
SQLite	2.9847	0.0885	0.0018	3.075

¹²As there are then 2 formulas to ground plus more solving overhead.

(Bs) confirms the interpretations of the (A1Bs) case: (L) and (N) perform similarly, (D) faster than both, now by a factor 20 to 40 because the query involved is slightly more complicated. Instances larger than the 1440 one, however, crashed the system altogether, because of lack of resources.

Also (Bf) shows results similar to (A1Bf), with (D) grounding efficiently but taking time to set up the database¹³, and the converse for (L). Both still outperform (N), (D) significantly so, (L) only just. For the instance on 490 constants, the first on which the naive version timed out:

db-system	parsing	grounding	solving	total
DuckDB	4.2826	0.305	0.0495	4.6371
SQLite	2.5049	434.7886	0.038	437.3315

(A3)	$\exists x \in A : \exists y \in B : P(x, y) \wedge q(x, y)$
	Should avoid a lot of grounding work in case P is sparse.
(Bt)	Sparse P ($\approx 1\%$ of $A \times B$), but $\neg q(x, y)$ known to hold for all of $A \times B$, making the theory trivially UNSAT.
(Bs)	Sparse P ($\approx 1\%$ of $A \times B$).
(Bf)	Full P (all of $A \times B$).

In the table below, we will summarise the (rounded) run times for the three subcases, all on the 1000-constant instance. Totals have been omitted.

	(Bt)			(Bs)			(Bf)		
db-system	prse	grnd	solve	prse	grnd	solve	prse	grnd	solve
Naive	0.17	241.18	21.33	0.23	301.16	17.89	7.07	3.67	18.90
DuckDB	18.47	53.69	0.01	0.24	0.20	0.08	14.66	3.00	4.31
SQLite	2.18	0.14	0.00	0.16	0.025	0.08	6.73	3.51	3.86

For (Bt) and (Bs), the differences between them for DuckDB, are that for the former case, quite some time is taken to register $\neg q(x, y)$ (for all constants) in the database, and to query for these facts while grounding, as we established before.

For SQLite, however, there is hardly any difference between the two, and querying goes extremely fast. This is because the formula $\exists x : \exists y : P(x, y) \wedge q(x, y)$ is already in the required standard form (refer to Section 3.3.3), and the query for the left-hand side of the conjunction is

¹³In the larger instances this is nearly all of the time compared to grounding and solving!

```
SELECT A.A, B.B FROM A,B WHERE
      (A.A,B.B) IN (SELECT P.A,P.B FROM P)
```

Unlike the more complex query in the (A1) case, the SQLite query optimiser recognises that $A \times B$ should not be generated, it simply queries for (and from) the table P .¹⁴

For **(Bf)**, it is no surprise that neither of the systems presents an improvement over the naive method, as $P = A \times B$ and q needs to be grounded over this full Cartesian product, no matter what we do.

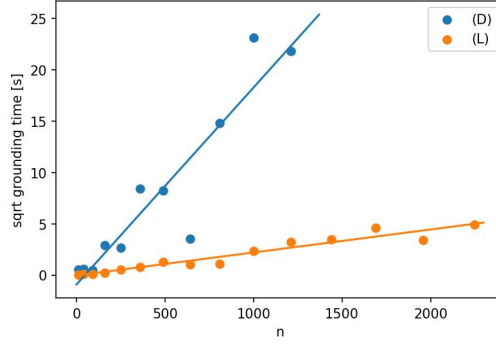
(A4)	$\exists x \in A : \exists y \in B : P(x, y) \wedge (Q(x, y) \vee f(x) = y)$
Should avoid a lot of grounding work in case P is sparse.	
(Bt)	$P = A \times B$, $Q = \emptyset$, but $f(x)$ is known to hold on exactly one of the tuples $\in P$, making the theory trivially SAT.
(Bs)	Sparse P ($\approx 1\%$ of $A \times B$) and not overlapping with Q ($\approx 50\%$ of $A \times B$), forcing $f(x) = y$ as part of the grounding.
(Bf)	Dense P ($\approx 95\%$ of $A \times B$) and empty Q , forcing $f(x) = y$ as part of the grounding.

For the **(Bt)** case, where we have to query over the full¹⁵ $A \times B$, the structure additionally contains a formula $f(a_i) = b_j$, for randomly generated indices i, j between 0 and $n - 1$. The randomness makes consistent comparing trickier, as it is not clear “where” in $A \times B$ the triviality occurs, but it does make the analysis below more relevant. (N) cannot profit from spotting the triviality early, so its solving time is much longer than that of (D) and (L). Because of this, (L) outperforms (N). (D) doesn’t because its grounding time is larger. The graph below shows the square root of only the grounding times for both (their parsing times were similar). It is clear that (D) loses most time querying every single time for the potential fact on the function f , timing out on the 1440 instance. The large variability around the best-fitting line¹⁶ can be explained by the randomness in *where* in $A \times B$ the triviality is found (because of the randomly generated i and j), and therefore *when* grounding can halt and SAT be returned.

¹⁴Refer to <https://www.sqlite.org/optoverview.html>, §2. WHERE Clause Analysis, showing that the optimiser will indeed recognise **IN (subquery)** but not **NOT IN (subquery)** (like we had in the (A1) case).

¹⁵As $P = A \times B$ and $Q = \emptyset$, the only non-trivial formula to ground is $\exists x \exists y P(x, y) \wedge \neg Q(x, y) \wedge (f(x) = y)$.

¹⁶A *line*, because we plot the square root of the times while the result set of the query has size n^2 .



The **(Bs)** results align with results we found earlier. Note that while we know that Q is irrelevant when grounding, and we should only look at the result set of P instead of $P \cap \overline{Q}$, it is a distraction for (L). This is confirmed by the times for the 1000-constant instance (on which (N) timed out):

db-system	parsing	grounding	solving	total
DuckDB	7.5521	0.3264	0.1143	7.9928
SQLite	3.5572	154.1538	0.093	157.804

As expected, for **(Bf)**, the conclusions are the same as for the (A3Bf) case. As P nearly covers all of $A \times B$, querying does not add a lot, as the naive grounding (with redundancies) is only slightly larger than the non-naive grounding.

(A5)	$\forall x \in A : \exists y \in B : P(x, y) \vee q(x, y)$
Should avoid a lot of grounding work in case P is dense.	
(Bt)	Ensuring that for every x , indeed exactly one y exists such that $P(x, y)$, will make the theory trivially SAT.
(Bs)	Sparse P : for $\approx 5\%$ of x 's, $P(x, y)$ holds on all the y 's, forcing $q(x, y)$ as part of the grounding.
(Bf)	Dense P : for $\approx 95\%$ of x 's, $P(x, y)$ holds on all the y 's, forcing $q(x, y)$ as part of the grounding, but keeping it small.

For the **(Bt)** case, both (D) and (L) spot the triviality smoothly and solve all instances in less than 2 seconds, while (N) took nearly 8 minutes for the 2250-constant instance.

For the **(Bs)** case, predictably, system (L) struggles with obtaining the large SAT-set of $\neg P$ and (using the terminology of Section 3.3.4) finding A_{NT} , and this overhead implies it is similar to (N). System (D) does better, solving the 2250-constant instance just in time. In the table below are their timings on the 640-constant instance, the last one on which all three finished.

db-system	parsing	grounding	solving	total
Naive	0.2406	198.8906	9.5194	208.6506
DuckDB	0.4341	14.5199	1.7126	16.6666
SQLite	0.2725	219.2629	1.6886	221.224

In the **(Bf)** case, (N) and (L) show times similar to their ones of the (Bs) case, while (D) is a lot quicker: on the 640-instance a parsing time of 7.1 and a grounding time of 1.7. This again supports two suspicions raised earlier. Firstly, that DuckDB takes more time than SQLite to set up the database. And secondly that SQLite loses the same amount of time querying for $\neg P$, even if now, in comparison to (Bs), the result set is much smaller (proving again that *getting* the result set is the bottleneck, not *iterating over* it). The only overhead for (D) is caused by iterating over the result set (while querying time is kept low), leading to a lower grounding time in this dense (Bf) case.

(A6)	$\forall x \in A : \exists y \in B : P(x, y) \wedge (r(x, y) \vee Q(x, y))$
	Should avoid a lot of grounding work in case P and Q are sparse. Note that we have to ensure that for every x , $P(x, y)$ holds for at least one y to make the theory SAT.
(Bt)	P and Q cover $\approx 50\%$ of $A \times B$ each but overlap, for all x , on exactly one constant y , making the theory trivially SAT.
(Bs)	Sparse P and Q , but disjoint and thus avoiding triviality. However, r is known to hold on the same tuples of constants as P , making the theory SAT.
(Bf)	Dense P and Q ($\approx 50\%$ of $A \times B$ each), but disjoint and thus avoiding triviality, forcing $r(x, y)$ as part of the grounding.

In the **(Bt)** case, $P(a_i, b_j)$ holds for all indices i and even j , $Q(a_i, b_j)$ holds for all i and odd j plus one randomly generated¹⁷ even index j . Both system (D) and (L) perform massively better than (N), which times out on the 490-constant instance already. Run times on the 2250-constant instance reveal again that DuckDB takes more time than SQLite to set up the database, while its grounding time remains almost constant over all the sizes of the instances.

db-system	parsing	grounding	solving	total
DuckDB	77.7009	0.7809	0.0053	78.4871
SQLite	36.4775	4.9998	0.0019	41.4792

Note that both take longer than the (A5Bt) case to spot the triviality: (D) to set up the fuller database, (L) to query for the more complex $P \cap Q$ which makes $A_{NT} = \emptyset$ and the theory trivially SAT.¹⁸

¹⁷Different for every i .

¹⁸Note that there is no negation involved in querying for $P \cap Q$, so this is – in line with what is remarked in footnote 14 – not forbiddingly complex for SQLite.

For the **(Bs)** case, $P(a_i, b_i)$ and $Q(a_i, b_{i+1})$ hold on all indices, so sparse structures but avoiding triviality. Through fact queries, we know that additionally, $r(a_i, b_i)$ holds as well. Predictably, (N) cannot profit from the extra information, inflating its solving times. Both (D) and (L) Perform extremely well. For the 1690-constant instance, right before (N) times out:

db-system	parsing	grounding	solving	total
Naive	0.138	300.9146	147.8965	448.9491
DuckDB	0.2391	0.7084	0.107	1.0545
SQLite	0.8551	1.6017	0.0957	2.5525

The reason for the excellent performance of both, is that this case avoids all their bottlenecks previously discussed. For DuckDB there is firstly not a lot of overhead to set up the database, as the predicates are sparse, and (therefore) secondly also not a lot of facts to be queried for when iterating. For SQLite, the potential bottleneck of querying can be restricted to *first*¹⁹ the sparse P , and only then intersecting this with Q .

In the **(Bf)** case, P and Q hold on the same constants as in the (Bt) case, but now without the additional random tuple for Q . As is to be expected, run times are higher in comparison to (Bs), as now the (only) non-trivial S_j covers half of $A \times B$. For (D), apart from its expected overhead of setting up the database in the parsing phase, it now also takes more time than in the (Bs) case to process the non-trivial S_j , but not forbiddingly so: for again the 1690-constant instance, its parsing and grounding times are respectively 45 and 135 seconds. As is to be expected, (L) takes a lot more time, simply to query for S_j , to the extent that it is not much more efficient than (N).²⁰

(A7)	$\exists x \in A : \forall y \in B : P(x, y) \wedge q(x, y)$
	Should avoid a lot of grounding work in case P is sparse. Note that – apart from in (Bt) – for P , we have to ensure that such an a_i (on which P holds for all y) indeed exists, in order to make the theory SAT.
(Bt)	Witness a_i does not exist for P and the theory is trivially UNSAT.
(Bs)	P sparse and $q(x, y)$ known to hold on that same x as a witness for P , making the theory SAT.
(Bf)	Dense $P = A \times B$, with q unknown, forcing the large grounding of q .

In the **(Bt)** case, P is filled with all tuples (a_i, b_j) , for all $i, j = 0, \dots, n-1$, but with $j \neq k$, for a randomly generated k . In other words, while P is dense, it every time misses just one constant b_k to create a witness a_i for the existential quantifier.

¹⁹As the only non-trivial φ'_j formula – in the terms of Section 3.3.4 – is for $P \wedge \neg Q$.

²⁰(L) timing out on 640 constants, (N) on 490.

As is to be expected, system (L) takes a while to establish that A_{NT} is empty²¹, while (D) processes the result set of the query smoothly, as we can clearly notice in their running times on the 2250-constant instance. Both outperform (N). Note that when the naive grounding is presented to the solver, it still takes quite some time to establish the formula as UNSAT.

db-system	parsing	grounding	solving	total
Naive	39.9909	402.6088	39.548	482.1477
DuckDB	75.7793	0.6935	0.0059	76.4787
SQLite	35.0474	370.3449	0.0018	405.3941

In the **(Bs)** case, $P(a_i, b_i)$ holds for all indices, but additionally, for a random k , $P(a_k, b_j)$ holds for all j , ensuring a_k is a witness. Through fact queries, q holds on exactly the same constants as P . In this case, with the terminology of Section 3.3.5, $A_{NT} = \{a_k\}$.

Again, (L) struggles to find this, to the extent that it is not faster than (N), essentially trading solving time for grounding time. (D) does it more efficiently but loses (some) time querying for the facts about q to establish SAT. On the 1960-constant case, right before both (N) and (L) time out, we have the following timings:

db-system	parsing	grounding	solving	total
Naive	0.1372	394.4076	97.0464	491.5912
DuckDB	0.4309	6.7376	0.1869	7.3554
SQLite	1.3078	559.0511	0.1559	560.5148

We do remark that right after this $n = 1960$ instance, the overhead for DuckDB to query for the facts, starts to show more prominently, with a grounding time of 69 seconds for $n = 2250$ and 150 seconds for $n = 2560$.

For **(Bf)**, as we mentioned in Section 3.3.5, establishing sets $NT_B(x)$ for the (many!) candidate witnesses causes some overhead. This makes (L) overall roughly as efficient as (N). Additionally, as we noticed before, DuckDB struggles to repeatedly execute the many queries which are needed to simplify the formula, making system (D) much slower than (L). For the 490-constant instance, right before (D) times out:

db-system	parsing	grounding	solving	total
Naive	1.4356	0.7332	3.7483	5.9171
DuckDB	3.5184	249.8406	0.99	254.349
SQLite	2.3131	4.1982	1.2357	7.747

²¹As remarked before, because this involves querying for the SAT-set of $\neg P$.

(A8)	$\exists x \in A : \forall y \in B : P(x, y) \vee (r(x, y) \wedge Q(x, y))$
	Should avoid a lot of grounding work in case P and Q are sparse. Note that we need to ensure that there indeed exists at least one constant a_k such that all the y 's in $P(a_k, y) \cup Q(a_k, y)$ cover the entirety of B , in order to make the theory SAT.
(Bt)	$P(a_k, y)$ holds on all $y \in B$, for a certain a_k , making the theory trivially SAT.
(Bs)	Sparse P and Q , in the sense that the potential witnesses for the x cover only $\approx 1\%$ of A .
(Bf)	Dense but non-overlapping P and Q , with $P \cup Q = A \times B$.

In the **(Bt)** case, Q is empty and $P(a_k, y)$ holds on all $y \in B$ for a randomly generated a_k . System (D) spots the triviality efficiently. (L) is not a lot more efficient than (N), like in (A7) trading solving for grounding time. On the 1000-constant case, we find the following timings:

db-system	parsing	grounding	solving	total
Naive	0.1305	34.6744	33.9518	68.7567
DuckDB	0.123	1.1787	0.0044	1.3061
SQLite	0.1447	54.4403	0.0061	54.5911

For the **(Bs)** case, for all a_i with $i \equiv 0 \pmod{100}$, $P(a_i, b_j)$ holds on even j and $Q(a_i, b_j)$ on odd j . Similarly, for the **(Bf)** case, $P(a_i, b_j)$ holds on even j and $Q(a_i, b_j)$ on odd j , but now for all a_i , which are all “candidate witnesses” for the existential quantifier, making the grounding for r dense. We present the results for both below, on the 360-constant instance (the last one of which we have timings for all). Totals have been omitted.

	(Bs)			(Bf)		
db-system	parsing	grounding	solving	parsing	grounding	solving
Naive	0.155	7.6993	5.0727	1.1545	283.7219	4.3623
DuckDB	0.1269	0.5749	0.0285	2.3902	115.1893	0.33
SQLite	0.1355	5.5123	0.0258	0.8966	212.9685	0.2503

While (D) is more efficient in both cases, it is clear that the dense (Bf) does pose a challenge. (L) is only marginally faster than (N) in the sparse (Bs) case, mainly because it avoids the large grounding, which makes solving faster.

5.2.3 Overall Conclusion of the Internal Comparison

Having tested different kinds of formulas, we can conclude that usually at least one of our two systems (D) and (L), decisively outperforms grounding naively.²² Which one of the two depends on the database operations involved. In the table below we summarise which database manager performs best on certain tasks, with a bold “+” for a significant difference which may cause the system to perform several orders of magnitude faster.

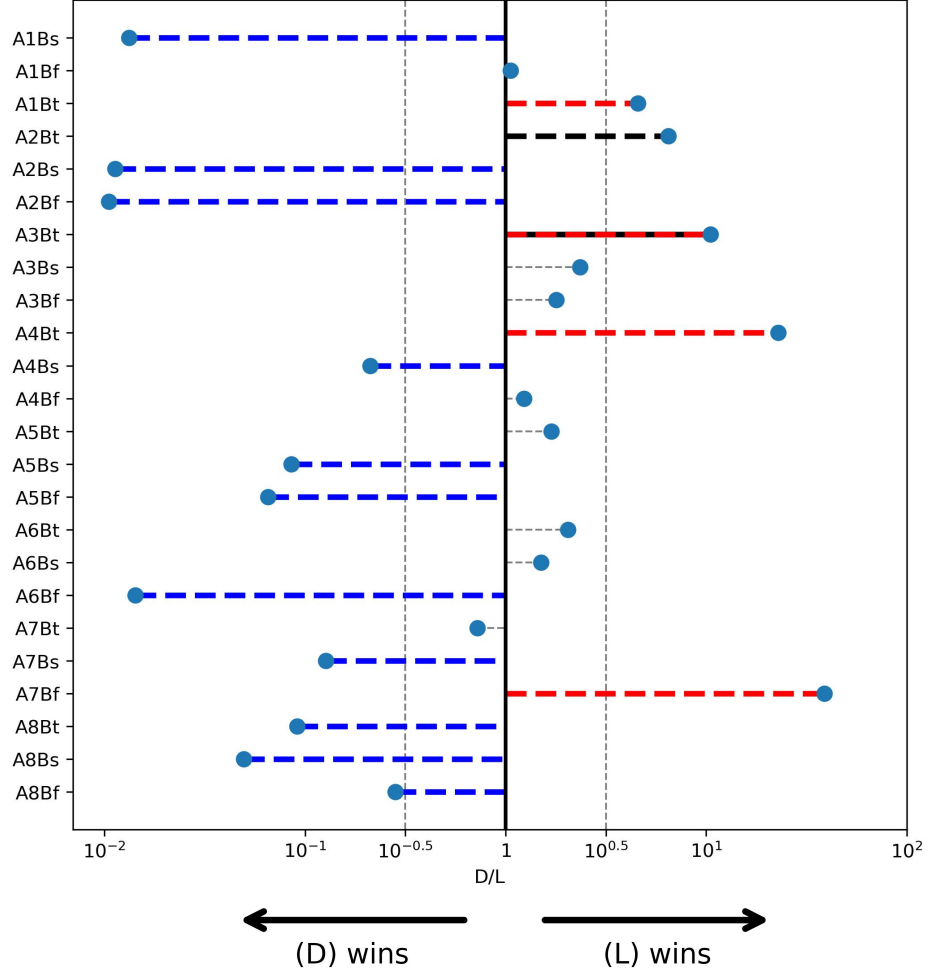
Database operation	Colour	DuckDB	SQLite
Inserting values into predicate tables (parsing + fact queries)	-----		+
Repeated querying for facts about predicates and functions	-----		+
Querying for SAT-sets of complex formulas (with negation(s))	-----	+	

The colours refer to the lines which are used in the plot below. The dots in this plot show, for all the cases, D/L on a logarithmic scale, with D and L the presolve timings (parsing + grounding) for respectively (D) and (L), on the 490-constant instance.²³ A dot at a large distance away from the vertical black line at 1, indicates a large difference between these timings. A dot on the right-hand side, at an x -coordinate of say $10^{1.2}$, means that system (D) took $10^{1.2}$ times longer than (L). Conversely, a dot on the left-hand side at say 10^{-2} , indicates that system (L) took 10^2 times longer than (D). The dashed lines away from the neutral ratio of 1, have been coloured in case there is a large²⁴ discrepancy to explain (with the colours mentioned in the table above).

²²Unless we have an edge-case where the configuration of the fully interpreted predicate(s) will cause them to behave similarly to naive grounding or even worse because of the extra overhead.

²³On which both systems finish on all cases, with the only exception of (L) on (A8Bf), which we re-ran with a timeout setting > 600 seconds.

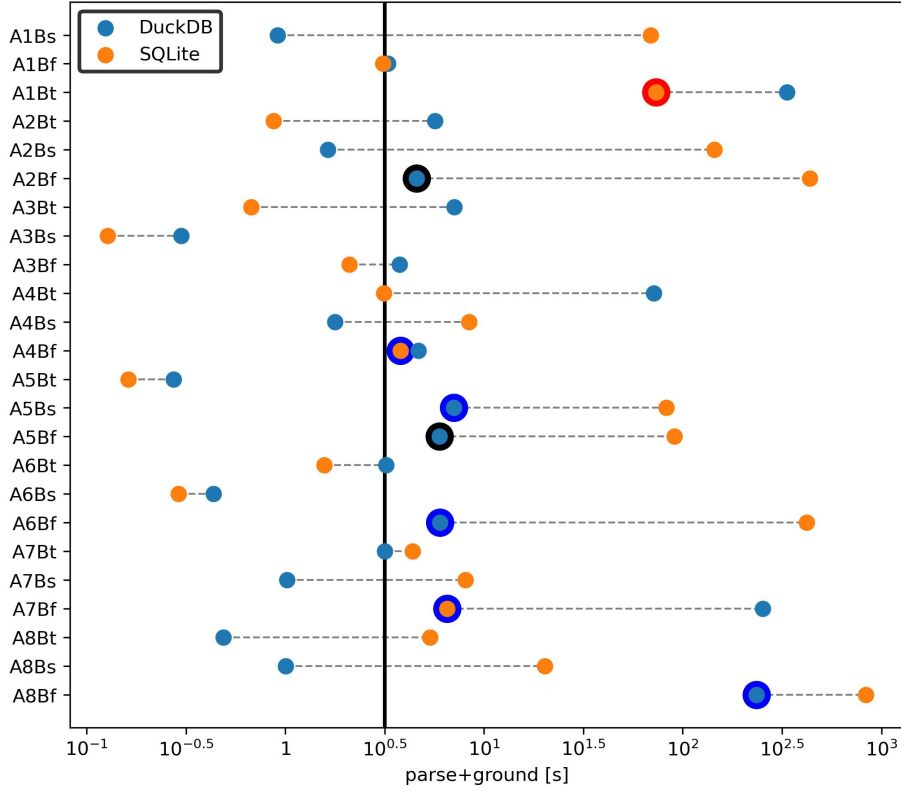
²⁴We arbitrarily interpret “large” as outside of the vertical grey lines in the plot at $10^{-0.5}$ and $10^{0.5}$, i.e. the ratio or its inverse larger than $\sqrt{10} \approx 3$.



Note that the plot above only gives insight into their *relative* performance, not their *absolute* timings. To compare these, below we plot the presolve timings of both (D) and (L), on again the 490-constant instance, also on a logarithmic scale. We include an explanation as to why the *best-performing of the two* still takes a long time, where we arbitrarily interpret “a long time” as a point to the right of $10^{0.5}$ (i.e. $\sqrt{10} \approx 3$ seconds). We colour-coded these as follows:

○	Grounding was inherently difficult because of the fully interpreted predicates in the structure. ²⁵ In these cases, bad performance needn’t necessarily worry us.
●	Bad performance is caused by setting up the database for a dense predicate. This is typically a bottleneck for DuckDB.
○	Bad performance is caused by having to query for a lot of facts in the database.

²⁵When grounding e.g. $\forall x (P(x) \Rightarrow q(x))$ for a dense P , whatever we do, we cannot avoid computing the conjunction of $q(x)$ over that large SAT-set of P .



Based on these plots and summarising the in-detail analysis of Section 5.2.2, we can conclude the following:

- DuckDB is by far the supreme database manager to query for SAT-sets, for which its longer setting up time is a relatively small price to pay.
- Repeatedly querying for facts causes a lot of overhead, especially for DuckDB, but does allow us to terminate early in case a trivial (UN)SAT situation can be spotted on the basis of this.
- Grounding a formula with a quantifier succession “ $\exists \forall$ ” over a dense structure, is a challenge for both (refer to case (A8Bf)), with the overhead of simplifying and projection sets, as already pointed out in Section 3.3.5.

't Is 'n schuun dingske geworden hé.

Prof. dr. Hendrik Van Maldeghem
(commenting on my finished thesis in 2014)

6

Conclusion

6.1 Improvements and Extensions

“Art is never finished, only abandoned.” A project like this is therefore always susceptible to improvement and extension.

Chapter 5 – and specifically Section 5.2.3 – suggest that one improvement could be to decide on using either DuckDB or SQLite based on heuristics of the problem to solve, e.g. whether there are fact queries which induce repeated querying (in which case SQLite would be the better choice) or whether we have to query for the SAT-sets of complex formulas (at which DuckDB is more efficient).

It could also be fruitful to attempt to alleviate the overhead for the “ $\exists \forall$ ” quantifier succession by attempting to find the sets involved (refer to Sections 3.3.5 and 4.3) more by means of SQL, as this has currently been implemented using Python lists.

One obvious extension is implementation of grounding formulas currently not supported in this system but the theory of which has been developed: alteration of more than 2 quantifier types and non-PNF formulas. Also extending the scope to larger portions of $\mathbf{FO}[\cdot]$ is an option, to support e.g. arithmetic and aggregates. Relevant questions there would be if (and how) built-in SQL aggregate functions such as `min`, `max`, `sum`, `count` etc. could be used to our advantage.

Another interesting direction to investigate, could be to check if (and how) the approach of using a database, is compatible with “non-standard” grounding techniques. One example is [De Cat et al., 2013], which discusses a technique called *incremental grounding*, in which formulas are grounded “on a needs basis”. One could e.g. write $\exists x \in A : \varphi(x)$ as $\varphi(x_0) \vee (\exists x \in A \setminus \{x_0\} : \varphi(x))$, and write the latter as $\varphi(x_0) \vee \psi$, keeping ψ ungrounded as long as $\neg\varphi(x_0)$ has not been shown to hold. While this approach does not seem incompatible at first glance, it deserves deeper

scrutiny. It could even allow us to relinquish the prerequisite of only being able to deal with finite domains.

Before extending the system, however, deeper testing and comparison of the existing system is needed as well, both “internally” and “externally”. For the mixed quantifier cases in Section 5.2,¹ we occasionally ran SLI on them, yielding run times similar to or longer than those of our system. While encouraging, those comparisons were not performed and analysed systematically enough to be included and discussed in the main report. A more systematic comparison and analysis, also with other solvers/grounders, would yield deeper insight into the workings and strengths and weaknesses of our system. Especially Logica (refer to [Skvortsov et al., 2024] and <https://github.com/EvgSkv/logica>), which also makes use of a RDBMS to represent a knowledge base, seems an interesting candidate, although it presents a fully integrated solver (with a less clear distinction between *parsing*, *grounding* and *solving*).

6.2 Final Conclusions

Representing the structure of a SAT-solving problem formulated in FOL over finite domains in a database, as explained in Chapter 1, proved to be a fruitful idea. Section 2.7 stated the aim of avoiding the “generate-then-simplify” approach to ground a theory, and we managed to demonstrate this can indeed be achieved by reformulating grounding as a database querying problem.

The contribution of this thesis is two-fold. The theoretical framework for database-assisted grounding has been presented (Chapter 3), and a working implementation of the system has been developed (Chapter 4).

Section 5.1 showed that both version (L) and (D) of the system can compete with a state-of-the-art grounder. The advantage of using a database is revealed most explicitly in case large amounts of trivially satisfied subformulas – that would be present in the naive grounding – can be avoided while grounding. In these cases, solving times are reduced drastically as well, by keeping the grounding small. Additionally, keeping track of facts about partially interpreted predicates and functions, allows the systems to potentially spot trivially (UN)SAT situations, and halt early.

Section 5.2 pointed out both versions’ strengths, weaknesses and differences. The experiments revealed that their efficiency depends on the kind of database operations involved. DuckDB proved more efficient for complex SAT-set queries, whereas SQLite performed better for repeated lightweight lookups of facts.

While the IDP-Z3-DB system is open to improvements and extensions, the results of this thesis indicate that using a relational database to assist grounding, is a promising approach.

¹(A5) to (A8).

Bibliography

- C. Barrett, P. Fontaine, and C. Tinelli. The SMT-LIB Standard: Version 2.7. Technical report, Department of Computer Science, The University of Iowa, 2025.
- A. Biere, H. van Maaren, T. Walsh, and M. Heule, editors. *Handbook of Satisfiability*. IOS Press, Feb. 2009.
- M. Brunsfeld et al. tree-sitter. URL <https://github.com/tree-sitter/tree-sitter>.
- P. Carbonelle and M. Denecker. FO[Core], a. URL <https://fo-dot.readthedocs.io/en/latest/Core.html>.
- P. Carbonelle and M. Denecker. FO[.] — FO-dot documentation, b. URL <https://fo-dot.readthedocs.io/en/latest/F0-dot.html>.
- P. Carbonnelle. IDP-Z3's Reference Manual. URL <https://docs.idp-z3.be/en/stable/index.html>.
- P. Carbonnelle. A Relational Theory of Grounding and a new Grounder for SMT, 2026. URL <https://arxiv.org/abs/2602.19102>.
- S. A. Cook. The complexity of theorem-proving procedures. In *Proceedings of the third annual ACM symposium on Theory of computing - STOC '71*, pages 151–158, Shaker Heights, Ohio, United States, 1971. ACM Press. doi: 10.1145/800157.805047. URL <http://portal.acm.org/citation.cfm?doid=800157.805047>.
- B. De Cat, M. Denecker, and P. Stuckey. Lazy Model Expansion by Incremental Grounding. *LIPICs, Volume 17, ICLP 2012*, 17:201–211, 2013. ISSN 1868-8969. doi: 10.4230/LIPICs.ICLP.2012.201. URL <https://drops.dagstuhl.de/entities/document/10.4230/LIPICs.ICLP.2012.201>.
- B. De Cat, B. Bogaerts, M. Bruynooghe, G. Janssens, and M. Denecker. Predicate Logic as a Modelling Language: The IDP System, Mar. 2018. URL <http://arxiv.org/abs/1401.6312>. arXiv:1401.6312.
- M. Denecker. Extending Classical Logic with Inductive Definitions, Mar. 2000. URL <http://arxiv.org/abs/cs/0003019>. arXiv:cs/0003019.
- DuckDB. URL <https://duckdb.org/>.
- IDP-Z3. URL <https://www.idp-z3.be/>.
- Microsoft Research. Z3 Guide. URL <https://microsoft.github.io/z3guide/docs/logic/intro/>.

- F. Niu, C. Ré, A. Doan, and J. Shavlik. Tuffy: scaling up statistical inference in Markov logic networks using an RDBMS. *Proceedings of the VLDB Endowment*, 4(6):373–384, Mar. 2011. ISSN 2150-8097. doi: 10.14778/1978665.1978669. URL <https://dl.acm.org/doi/10.14778/1978665.1978669>.
- SAT Competitions. The International SAT Competition Web Page. URL <https://satcompetition.github.io/>.
- E. Skvortsov, Y. Xia, S. Bowers, and B. Ludäscher. The Logica System: Elevating SQL Databases to Declarative Data Science Engines. In *Datalog 2.0 Workshop at ICLP 2024*. CEUR-WS.org, 2024.
- SQLite. URL <https://sqlite.org/index.html>.
- L. Van Laer. DIRT, a. URL <https://gitlab.com/EAVISE/sli/DIRT>.
- L. Van Laer. SLI, b. URL <https://gitlab.com/sli-lib/SLI>.
- L. Van Laer, S. Vandevelde, and J. Vennekens. Efficiently grounding FOL using bit vectors, Aug. 2024. URL <http://arxiv.org/abs/2408.07980>. arXiv:2408.07980.
- L. Van Laer, S. Vandevelde, and J. Vennekens. DIRT: a Literature-Based Benchmark Suite for Grounders. In G. Casini, B. Dundua, and T. Kutsia, editors, *Logics in Artificial Intelligence*, pages 343–356, Cham, 2025. Springer Nature Switzerland. ISBN 9783032045874. doi: 10.1007/978-3-032-04587-4_21.
- S. Vandevelde. IDP-Z3. URL https://gitlab.com/krr/IDP-Z3/-/tree/main?ref_type=heads.
- S. Vandevelde. Map Coloring, 2020. URL https://cdmn.readthedocs.io/en/stable/Examples/map_coloring.html#map-coloring.
- J. Vennekens. Building a reasoning engine for first-order logic with a RDBMS. URL <https://dinf.vub.ac.be/thesis/proposals/jv-topic4>.
- z3py Namespace Reference. URL <https://z3prover.github.io/api/html/namespacez3py.html>.
- L. Zhang and S. Malik. The Quest for Efficient Boolean Satisfiability Solvers. In E. Brinksma and K. G. Larsen, editors, *Computer Aided Verification*, pages 17–36, Berlin, Heidelberg, 2002. Springer. ISBN 9783540456575. doi: 10.1007/3-540-45657-0_2.
- M. Zuhra. Exploring DuckDB and the Columnar Advantage, 2023. URL <https://medium.com/@zujkanovic/exploring-duckdb-and-the-columnar-advantage-f7beb8cbf478>.